组合数学讲义

xzsawq

2026 年 7 月 18 日

前言

本文是笔者在讲授《具体数学》部分相关内容时整理的讲义。然而，本文的内容与《具体数学》鲜少重合。相比重复《具体数学》的内容，笔者更希望能够传达一些书上没有但是在 OI 中较为常见的方法和技巧。目前，OI 中较少直接考察纯粹的数学内容。然而，数学作为工具和语言无处不在。例如，在分析复杂度时常常会出现递归式和和式；在组合计数问题中，有大量的二项式系数出现；在刻画一个问题时，通过数学语言重新描述的问题往往会带来新的直观。诚然，许多问题都可以通过精巧的做法来绕过数学，但如果没有这样敏锐的观察力，也许就需要扎实的数学功底来一步步推平面前的障碍。事实上，许多其他的知识的学习也是如此。有很多的问题是“横看成岭侧成峰”的，如果你没有看到那个最容易的侧面，那么也许你就需要通过其他的角度解决问题，或是凭着扎实的基本功正面突破。笔者希望通过这门课程帮助同学们提升数学方面的基本功，从而在面对一些复杂的数学问题时也能自信地一步步将其解决。

还可以用推土机把山推平了或者用炸弹给炸掉。

由于时间和难度的原因，讲义无法面面俱到。由于语言的局限，讲义中可能有许多无法解释清楚的地方，也无法完美地传达笔者的所思所想。不过，笔者尽量补齐了课上讲过和希望讲的所有内容，并力图读者**在 AI 工具的辅助下**能够清楚和深入地理解本讲义的内容与其背后的**思路**。

本讲义并不完全按照学习的顺序来编写。例如，几乎每道题我们都给出了生成函数的方法，而生成函数本身要到最后才学习。因此，对于这类内容（以 ⧗ 标记），初学时应当暂时跳过，等到学完相关知识后回头再看。还有一些较为困难的内容（以 * 标记），初学时可能难以完全掌握，可以量力而为。

全部跳过。

《具体数学》是一本非常经典的著作，涵盖了非常丰富的内容。从本书中可以学到很多其他地方难以找到的方法和技巧，也可以从书中看到大师是如何思考问题的。另外，阅读本书可以很好地帮助 OIer 培养离散数学方面的“数学成熟度”。冯·诺依曼曾经说过，“In mathematics you don’t understand things. You just get used to them.”（在数学中，你并不理解事物，你只是习惯了它们）。习惯数学是非常重要的！《具体数学》直到 5.4 为止的内容都是对 OI 非常有用的。再往后的内容中，有些与 OI 关联不大，有些会在其他地方学到。但是，读一读仍然会有很大收获。

对于离散数学的初学者来说，《离散数学及其应用》也是一本不错的入门教材。笔者强烈建议至少把这本书的前两章看一看，其中有很多重要且基础的数学语言。后续的内容也都是和 OI 比较相关的内容，只不过相对比较基础，只需浏览一下即可。

在讲授本课程期间，笔者让同学们每周上课前自行阅读《具体数学》的对应章节并完成习题。在上课时，仅仅快速地过一遍教材并强调重点，然后开始讲解其他内容。在做书上的习题时，笔者认为不应该查阅前面的正文，以保证对所学知识的记忆。原则上，习题应该全部完成，但为了减轻工作量，只挑选选了一部分有代表性的题目作为书面任务。此外，还有一些补充的 OI 题目。数学习题应当与 OI 题按同样方式对待。笔者期望的课下任务量是每周 8-10 个小时（包含阅读和做题）。

我上学的时候从不做选做任务。

在遇到努力思考但仍然不会的题目时，有以下几种解决办法：

- （对《具体数学》）翻到附录 A 查看答案

- （对 OI 题）翻到题解或者网上搜索相关题目的题解
- 询问同学/老师/群友
  - 理想情况下，你应该表达清楚卡住的点（比如已经尝试过什么、到哪里做不下去了、或者干脆一点头绪没有等等），然后被提问者需要努力提供一个合适的提示。（但是不一定能及时回复）
  - 而现实中这样沟通的成本往往较高，且问题未必能得到及时的解答。
- 询问 AI：AI 工具的出现让上面的问题几乎不复存在了。可以让 AI 来充当上一条中的被提问者。只不过，你需要表达清楚你想让 AI 做的事情，并且需要自行判断 AI 回答的正确性。

现在还可以让 AI 批改自己的作业。参考提示词如下：帮我批改一下这份作业，题目是 xxx（后附作业内容）。另外，规范地使用 Markdown 或 LaTeX 是非常重要的，你可以向 AI 询问有哪些基本的规范，也可以通过 AI 来学习各种使用技巧。

本讲义的编写离不开课上同学们的反馈。大家提出了非常多的有价值的问题，使笔者可以认识到哪些内容是困难的、哪些内容是需要进一步澄清的，这使得讲义内容更加清晰、完善和易于理解。如果读者有任何问题或者建议，欢迎随时联系笔者（QQ：2103732004）。

什么时候智械危机？

目录

第一章 递归式 **5**

1.1 课前任务 5
1.2 写递归式 5
1.3 解递归式 6
1.4 “成套方法” 7

第二章 和式 **8**

2.1 课前任务 8
2.2 基本方法 8
2.3 扰动法 8
2.4 多重和式 9
2.4.1 三角求和的对称技巧 9
2.4.2 很多重求和 9
2.5 差分再求和 12
2.6 指示器变量 14
2.7 幂的处理 14
2.7.1 展开乘积 14
2.7.2 普通幂和下降幂的互相转换 15
2.8 拆括号容斥 16
2.9 综合练习 17
2.10 补充的 OI 题及解答 20

第三章 取整函数 **24**

3.1 课前任务 24
3.2 处理取整的一些技巧 24
3.3 整除分块 25
3.4 单位根反演 28
3.4.1 扩域 29
3.5 Kummer 定理 30
3.6 补充的 OI 题及解答 30

第四章 数论 **32**

4.1 课前任务 32
4.2 整除 32
4.3 素数 33
4.3.1 积性函数 34
4.4 取模和同余 35

4.4.1 逆元 35
4.4.2 中国剩余定理 36
4.4.3 同余等差数列 37

第五章 二项式系数和生成函数 38
5.1 课前任务 38
5.2 组合计数原理 38
5.2.1 加法、乘法、减法原理 38
5.2.2 商集与除法原理 38
5.3 经典组合计数模型 39
5.3.1 球与盒子 39
5.3.2 变量计数 44
5.3.3 排列 45
5.3.4 数列 46
5.3.5 格路径 47
5.3.6 卡特兰数 49
5.4 根据组合结构 dp 52
5.5 利用乘法原理 dp 52
5.6 生成函数初步 53
5.6.1 运算法则 I 53
5.6.2 导数 54
5.6.3 运算法则 II 54
5.6.4 最重要的一些生成函数 55
5.6.5 用生成函数处理序列 55
5.6.6 计算生成函数 56
5.6.7 更多技巧 57

1 递归式

本章提要

- 递归式的概念
- 寻找递归式的基本方法
- 数学归纳法
- 解递归式的一些方法

1.1 课前任务

《具体数学》的第一章中，最重要的内容有以下几点：递归式的概念、数学归纳法、如何写出递归式、“成套方法”。

挑选的部分习题：1 3 5 8 9 12 14 16

1.2 写递归式

练习 1.1. 对下面的问题写出递归式。事实上，大家在写 dp 的时候就在做这样的事情。

1. 骨牌覆盖：用 1×2 和 2×1 的骨牌覆盖一个 $2\times n$ 的网格，求方案数
2. 骨牌覆盖 2：用 1×2 和 2×1 的骨牌覆盖一个 $3\times n$ 的网格，求方案数
3. 长为 n 的 01 串，要求不能出现连续的三个 1，求方案数
4. 长为 n 的 01 串，要求不能出现连续的三个 1 或者连续的两个 0，求方案数
5. 用 m 种颜色给 $1\times n$ 的网格涂色，要求相邻格子颜色不相同
6. 长为 $2n$ 的合法括号序列的数量
7. 从 $(0,0)$ 走到 (n,n)，每步可以向上或向右走，不能跨过对角线的方案数
8. n 选 k 排列数
9. n 选 k 组合数
10. 把 n 写成若干单调不降正整数之和的方案数（hint: 你也许需要添加一维）
11. n 个人围成一圈，选若干个人，要求不能相邻，求方案数
12. n 个数的排列，要求 $a_i\neq i$，求方案数

课上其实一道题都没做。

评注. 你还能提出其他的问题吗？你能推广上面的某些问题吗？（无需会做）

解答. TODO

评注. 下面是一些可能的推广：

- 骨牌覆盖：用 1×2 和 2×1 的骨牌覆盖一个 $n\times m$ 的网格，求方案数
- 长为 n、字符集大小为 m 的字符串，要求某些子串不能出现，求方案数
- 从 $(0,0)$ 走到 (n,m)，每步可选一个集合内的一种，另有对不能经过的位置的限制，求方案数

- 把 n 写成若干单调不降正整数之和的方案数，额外要求：严格递增/数都不超过 k/数都至少是 k/必须是奇数/...
- n 个数的排列，要求恰好 k 个位置 $a_i \neq i$，求方案数

1.3　解递归式

我们有两种目的：得到一个**封闭形式**，或者**快速计算**。对于只有一个递归项的递推式 $f_n = a_n f_{p(n)} + g(n)$，可以考虑展开变成求和：$f(n) = g(n) + a_n g(p(n)) + a_n a_{p(n)} g(p(p(n))) + \cdots$。

例 1.2. 展开递推式

$$J(n) = 3J(\lfloor n/2 \rfloor) + \gamma n + a_{n \bmod 2}.$$

解答.

$$\begin{aligned} J(n) &= 3J(\lfloor n/2 \rfloor) + \gamma n + a_{n \bmod 2} \\ &= 3\left(3J(\lfloor n/4 \rfloor) + \gamma \lfloor n/2 \rfloor + a_{\lfloor n/2 \rfloor \bmod 2}\right) + \gamma n + a_{n \bmod 2} \\ &= 3^2 J(\lfloor n/4 \rfloor) + 3\gamma \lfloor n/2 \rfloor + 3a_{\lfloor n/2 \rfloor \bmod 2} + \gamma n + a_{n \bmod 2} \\ &\cdots \\ &= 3^k J(\lfloor n/2^k \rfloor) + \sum_{i=0}^{k-1} 3^i \left(\gamma \lfloor n/2^i \rfloor + a_{\lfloor n/2^i \rfloor \bmod 2}\right). \end{aligned}$$

假设 n 在二进制下表示为 $(b_{k-1} b_{k-2} \cdots b_0)_2$，即 $n = \sum_{i=0}^{k-1} b_i 2^i$。那么我们可以将上式写为：

$$\begin{aligned} J(n) &= 3^k J(0) + \sum_{i=0}^{k-1} 3^i \left(\gamma \left\lfloor \frac{n}{2^i} \right\rfloor + a_{\left\lfloor \frac{n}{2^i} \right\rfloor \bmod 2}\right) \\ &= 3^k J(0) + \sum_{i=0}^{k-1} 3^i \left(\gamma \sum_{j=i}^{k-1} b_j 2^{j-i} + a_{b_i}\right) \\ &= 3^k J(0) + \gamma \left(\sum_{j=0}^{k-1} b_j 2^j \sum_{i=0}^{j} \left(\frac{3}{2}\right)^i\right) + \sum_{i=0}^{k-1} 3^i a_{b_i} \\ &= 3^k J(0) + \gamma \left(\sum_{j=0}^{k-1} b_j \frac{3^{j+1} - 2^{j+1}}{3-2}\right) + \sum_{i=0}^{k-1} 3^i a_{b_i} \end{aligned}$$

对于形如 $f_n = \sum_{i=1}^{k} a_i f_{n-i} + r^n P(n)$（其中 $P(n)$ 是多项式）的递推，可以参考**常系数线性递推**理论。不过，也许大家更熟悉的方法是写成矩阵乘法然后快速幂，这是一个稍劣一些的做法。对于多个相互依赖的线性递推，可以考虑生成函数，也可以写成矩阵乘法然后快速幂。

对于二维的递推，可以尝试的做法是对其中一维建立生成函数，然后转为生成函数的一维递推。但这通常也很困难。对于分式类型的递推（如 $f_n = \frac{1}{f_{n-1}+1}$），可以尝试将 f_n 写为 p_n/q_n，然后导出对应的递推。其他的递推往往很难解。　人生多艰。

1.4　“成套方法”

事实上，下面的思想往往更有用：在处理非齐次递推（如 $f_{n,m}=f_{n-1,m}+f_{n,m-1}+g(n,m)$）时，可以转而考虑每个非齐次项 $g(i,j)$ 和初值项 $f_{0,\cdot},f_{\cdot,0}$ 对最终的 $f_{n,m}$ 的**贡献系数**。这种“算贡献”的思想对解决其他问题常常也有帮助。

例 1.3. 对于递推式 $f_{n,m}=f_{n-1,m}+f_{n,m-1}+g(n,m)$，求出 $f_{n,m}$ 关于 $g(i,j)$、$f_{0,j}$ 和 $f_{i,0}$ $(1\le i\le n,1\le j\le m)$ 的表达式。

解答. 考虑 $g(i,j)$ 对 $f_{n,m}$ 的贡献系数 $\gamma(n,m)$，这是令 $g(i,j)=1$、其他的 g 和 f 的初值都为 0 时 $f_{n,m}$ 的值。通过平移，可以知道这就是 $h_{x,y}=h_{x-1,y}+h_{x,y-1}+[x=y=0]$ 在 $(n-i,m-j)$ 处的值。通过组合意义，可以发现 $h_{x,y}$ 就是 $(0,0)$ 向右向上走到 (x,y) 的方案数。所以 $g(i,j)$ 对 $f_{n,m}$ 的贡献系数为 $\binom{n-i+m-j}{n-i}$。

初值 $f_{i,0}$ 和 $f_{0,j}$ 的贡献系数可以类似考虑，只不过要小心一点：我们只有 $n,m\ge 1$ 时才会调用上面的递推式进行计算，否则直接返回初值。所以，这里的贡献系数分别是 $h_{n-i,m-1}$ 和 $h_{n-1,m-j}$。

把上面的结果合到一起，我们就得到了

$$f_{n,m}=\sum_{i,j\ge 1}g(i,j)\binom{n-i+m-j}{n-i}+\sum_{i=1}^{n}f_{i,0}\binom{n-i+m-1}{n-i}+\sum_{j=1}^{m}f_{0,j}\binom{n-1+m-j}{n-1}.$$

本章总结

- 使用数学归纳法证明结论
- 写递归式：分离“最后一个”或者“最特殊的一个”元素
- 解递归式：直接展开求解；计算贡献系数求解

2 和式

本章提要

- 求和符号以及艾弗森括号的使用
- 化简和式的基本方法
- 化简和式的常见技巧
- 下降幂和相关结果

2.1 课前任务

阅读具体数学《第二章》。在本章中最重要的内容有以下几点：求和符号以及艾弗森括号的使用、扰动法、三角求和的对称技巧、化简和式的一些常见方法、下降幂和有限微积分。

挑选的部分习题：2 3 4 6 13 14 19 20 21 22 25 32；7 9 10 11 17 23 24 27 29 31；35* 36*

这是两周的量

2.2 基本方法

处理和式是基本中的基本。我们在书中、平时做题中、本系列课程中会遇到非常多的和式。下面是化简和式的一些基本方法：

- 尽量使用简单的求和条件
- 把无关项用分配律提出到求和号外
- （通过拆成多个求和）让求和项只含有乘积
- 通过换元让下标保持简单
- 交换求和号常常是有用的

2.3 扰动法

例 2.1. 数列求和.

$$\sum_{i=1}^{n} i^k a^i.$$

需要 $O(k^2)$。

这是个很好的练习。

解答. 设 $S_n = \sum_{i=1}^{n} i^k a^i$，那么首先

$$S_n = \sum_{i=1}^{n} (i+1)^k a^{i+1} - (n+1)^k a^{n+1} + 1^k a^1.$$

扰动法的关键一步在于把 $(i+1)^k a^{i+1}$ 表示成关于 i 而不是 $i+1$ 的表达式。在遇到和的幂时，我们的第一反应就是二项式定理：

$$(i+1)^k a^{i+1} = a \sum_j \binom{k}{j} i^j a^i.$$

把这个带到扰动法中就得到

$$S_n = \sum_{i=1}^{n} a \sum_{j} \binom{k}{j} i^j a^i = a \sum_{j} \binom{k}{j} \sum_{i=1}^{n} i^j a^i.$$

（要牢记我们的基本准则“提出无关项”和“交换求和号”）。

现在，我们发现内部的求和和 S_n 很像，只是 k 换成了 j。这种时候我们往往可以尝试设出辅助递归项 $S_{n,j}$，而 $S_{n,j}$ 关于 j 的递归具有同样的模式。

特别地，在 $a = 1$ 时无法得到 $S_{n,k}$ 的方程，而是 $S_{n,k-1}$ 的方程。然而，这并不影响我们能够递推求解 $S_{n,j}$ 的事实。这样我们就得到了 $O(k^2)$ 的做法。

另解. 我们也可以先写成递推式

$$S_n = S_{n-1} + n^k a^n,$$

然后挥动线性递推重锤。这样甚至可以得到一个 $O(k)$ 的做法。具体来说：　并非具体

- 线性递推告诉我们结果具有 $a^n P(n) + C$ 的形式，其中 $P(n)$ 是一个 k 次多项式，C 是常数
- 先递推计算出 $S_0, \ldots, S_k$，然后 $P(i) = a^{-i}(S_i - C)$
- 使用 Lagrange 插值和直接递推两种方式分别计算 $P(k+1)$，从而解出 C
- 使用 Lagrange 插值得到 $P(n)$ 和 S_n

例 2.2. 含有求和的递归. 寻找 $s_n = \sum_{i=1}^{n} f_i$ 的递归式，其中 f_n 是 Fibonacci 数列

解答. 邻项相减/扰动法/生成函数

2.4　多重和式

2.4.1　三角求和的对称技巧

在三角求和的例子中，我们利用对称性把 $i \le j$ 的求和变成无限制的求和以及 $i = j$ 的求和。这是常用的技巧。

例 2.3. 给定平面上 n 个点，求所有点对的曼哈顿距离之和。也就是计算

$$\sum_{1 \le i < j \le n} |x_i - x_j| + |y_i - y_j|.$$

例 2.4. 异或粽子

在某种意义上，这个技巧可以推广为 Burnside 引理。　推得太广了。

2.4.2　很多重求和

我们来看一下下面这个 m 重求和。

例 2.5.

$$\sum_{1\le i_1\le i_2\le\cdots\le i_m\le n} 1$$

是多少？

我们有很多种方式来推导它：找规律后归纳、组合意义、有限微积分、写递归式，等等

方法一：找规律

打表发现答案是 $\binom{n+m-1}{m}$。如何证明呢？可以按 $n+m$ 归纳：

$$\begin{aligned}
&\sum_{1\le i_1\le\cdots\le i_m\le n} 1\\
&=\sum_{1\le i_1\le\cdots\le i_m<n} 1+\sum_{1\le i_1\le\cdots\le i_{m-1}\le i_m=n} 1\\
&=\binom{(n-1)+m-1}{m}+\binom{n+(m-1)-1}{m-1}\\
&=\binom{n+m-1}{m}.
\end{aligned}$$

$n=1$ 或 $m=1$ 的情况是显然的。

方法二：组合意义

我们可以编出很多种组合意义：

- 对每个 $t=1,2,\ldots,n$，考虑有多少个求和指标等于 t。这样相当于把 m 个球放到 n 个可空的桶里
- 或者，考虑 $[1,i_1],(i_1,i_2],(i_2,i_3],\ldots,(i_{m-1},i_m],(i_m,n]$ 这 $m+1$ 个桶，其中第一个桶非空，其他可空。
- 再或者，每个 $1\le i_1\le i_2\le\cdots\le i_m\le n$ 一一对应一条路径 $(1,1)\to(1,i_1)\to(2,i_1)\to(2,i_2)\to(3,i_2)\to(3,i_3)\to\cdots\to(m,i_m)\to(m+1,i_m)\to(m+1,n)$ 的路径，其中 $(t,i_t)\to(t+1,i_t)$ 是一个向右的步，$(t,x)\to(t,x+1)$ 是一个向上的步。那么相当于从 $(1,1)$ 出发向右向上走到 $(m+1,n)$

方法三：有限微积分

这不过是 $\sum i^{\overline{k}} = \frac{i^{\overline{k+1}}}{k+1}$ 的反复应用：

$$\begin{aligned}\sum_{1\le i_1\le\cdots\le i_m\le n} 1 &= \sum_{i_m=1}^{n}\sum_{i_{m-1}=1}^{i_m}\cdots\sum_{i_1=1}^{i_2} 1 = \sum_{i_m=1}^{n}\sum_{i_{m-1}=1}^{i_m}\cdots\sum_{i_2=1}^{i_3} i_2\\ &= \sum_{i_m=1}^{n}\sum_{i_{m-1}=1}^{i_m}\cdots\sum_{i_3=1}^{i_4}\frac{i_3^{\overline{2}}}{2}\\ &= \sum_{i_m=1}^{n}\sum_{i_{m-1}=1}^{i_m}\cdots\sum_{i_4=1}^{i_5}\frac{i_4^{\overline{3}}}{2\cdot 3}\\ &\vdots\\ &= \sum_{i_m=1}^{n}\frac{i_m^{\overline{m-1}}}{(m-1)!} = \frac{n^{\overline{m}}}{m!} = \binom{n+m-1}{m}.\end{aligned}$$

方法四：写递归式

$$\begin{aligned}f_{m,n} &= \sum_{1\le i_1\le\cdots\le i_m\le n} 1\\ &= \sum_{i_m=1}^{n}\sum_{1\le i_1\le\cdots\le i_{m-1}\le i_m}\\ &= \sum_{i_m=1}^{n} f_{m-1,n}.\end{aligned}$$

差分就得到 $f_{m,n} - f_{m,n-1} = f_{m-1,n}$。或者，在一开始就考虑

$$\begin{aligned}f_{m,n} &= \sum_{1\le i_1\le\cdots\le i_m\le n} 1\\ &= \sum_{i_m=1}^{n}\sum_{1\le i_1\le\cdots\le i_{m-1}\le i_m<n} 1 + \sum_{i_m=1}^{n}\sum_{1\le i_1\le\cdots\le i_{m-1}\le i_m=n}\\ &= f_{m,n-1} + f_{m-1,n}.\end{aligned}$$

那么如何解这个递推式呢？这是我们先前考虑过的例子。代入就得到

然而之前这个例子是通过组合意义做的。

$$f_{m,n} = \sum_{i=1}^{m}\binom{n+m-i-1}{m-i} f_{i,0} + \sum_{j=1}^{n}\binom{n+m-j-1}{n-j} f_{0,j}.$$

这个初值应该是多少呢？$f_{i,0}$ 显然应该是 0，而 $f_{0,j}$ 的合理选择应该是 1，因为 $j = f_{1,j} = f_{0,j} + f_{1,j-1} = f_{0,j} + (j-1)$。这样我们就得到

$$\begin{aligned}f_{m,n} &= \sum_{j=1}^{n}\binom{n+m-j-1}{n-j}\\ &= \sum_{j=0}^{n-1}\binom{m+j-1}{j}\\ &= \binom{n+m-1}{m}.\end{aligned}$$

可是我们还没有学习组合数的处理

方法五：生成函数

TODO：我们在第五章再来处理这个问题

2.5　差分再求和

我们接下来要介绍的重要技巧是**差分再求和**，也就是把 $f(i)$ 写成 $f(0)+\sum_{k=1}^{i}(f(k)-f(k-1))$。然后通过交换求和号，我们往往能够对新的和式进行简化。

注意这里 i 需要是自然数

例 2.6.

$$i=\sum_{k=1}^{i}1=\sum_{k\geq 1}[i\geq k].$$

评注. 如果套上一层期望，我们就得到 $\mathbb{E}[X]=\sum_{k\geq 1}\Pr[X\geq k]$。

例 2.7. 分部求和法.

$$\begin{aligned}\sum_{i=1}^{n}a_ib_i&=\sum_{i=1}^{n}a_i\left(b_0+\sum_{j=1}^{i}(b_j-b_{j-1})\right)\\&=b_0\sum_{i=1}^{n}a_i+\sum_{j=1}^{n}(b_j-b_{j-1})\sum_{i=j}^{n}a_i\\&=b_0\sum_{i=1}^{n}a_i+\sum_{j=1}^{n}(b_j-b_{j-1})\left(\sum_{i=1}^{n}a_i-\sum_{i=1}^{j-1}a_i\right)\\&=b_n\sum_{i=1}^{n}a_i-\sum_{j=1}^{n}(b_j-b_{j-1})\sum_{i=1}^{j-1}a_i.\end{aligned}$$

注意以上的第二行和第四行都是可用的结论。也就是说，对 a_ib_i 求和可以转为对 $(\sum a_i)(\Delta b_i)$ 求和。

练习 2.8. 数列求和再放送.

$$\sum_{i=1}^{n}i^ka^i.$$

使用分部求和法得到递推式，据此同样可以得到 $O(k^2)$ 的做法。

这也是个很好的练习。

接下来的一个重要应用是对 $f(\min(a_1,\ldots,a_n))$ 或 $f(\max(a_1,\ldots,a_n))$ 的求和。

例 2.9.

$$\begin{aligned}\sum_a f(\min(a_1,\cdots,a_n)) &= \sum_a \left(f(0)+\sum_{i=1}^{\min(a_1,\cdots,a_n)}(f(i)-f(i-1))\right)\\ &= \left(\sum_a f(0)\right)+\sum_a\sum_{i\geq 1}[\min(a_1,\cdots,a_n)\geq i](f(i)-f(i-1))\\ &= \left(\sum_a f(0)\right)+\sum_{i\geq 1}(f(i)-f(i-1))\sum_a[\forall k,\ a_k\geq i].\end{aligned}$$

同理可以得到

$$\sum_a f(\max(a_1,\cdots,a_n)) = \left(\sum_a f(0)\right)+\sum_{i\geq 1}(f(i)-f(i-1))\sum_a[\exists k,\ a_k\geq i].$$

这样可以把最值求和转成判定问题来计数。

评注. 最常见的应用是 $f(x)=x$。请你在上面的结果中代入它并熟悉一下得到的结果。

练习 2.10. 给一棵树，点有点权，求所有路径 min 的 k 次方的和。

解答.

$$\begin{aligned}&\sum_{\text{路径}P}\min(P)^k = \sum_{\text{路径}P}\sum_{i=1}^{\min(P)} i^k-(i-1)^k\\ &=\sum_{i\geq 1}(i^k-(i-1)^k)\sum_{\text{路径}P}[\min(P)\geq i]\\ &=\sum_{i\geq 1}(i^k-(i-1)^k)\sum_{\text{路径}P}[P\text{ 是只保留权值}\geq i\text{ 的点形成的森林中的一条路径}]\\ &=\sum_{i\geq 1}(i^k-(i-1)^k)\sum_B[B\text{ 是只保留权值}\geq i\text{ 的点形成的森林中的一个连通块}]\binom{|B|+1}{2}\end{aligned}$$

假设点权从大到小为 $w_m,\ldots,w_1$。在 $i\in[w_i,w_{i+1})$ 时第二个求和的结果都相同，而 i 从 w_i 减小到 w_i-1 时连通块会发生合并。并查集维护连通块大小即可。

另解.　也可以直接枚举 min 的取值：

$$\begin{aligned}\sum_{\text{路径}P}\min(P)^k &= \sum_w w^k\sum_{\text{路径}P}[\min(P)=w]\\ &= \sum_w w^k\sum_{\text{路径}P}([\min(P)\geq w]-[\min(P)>w]\end{aligned}$$

剩下的过程是一样的。通过分部求和法也不难发现这两个式子相等。

练习 2.11. 给一个 0 到 $n-1$ 的排列，求所有区间的 mex 的 k 次方的和。$\operatorname{mex}(S)$ 为 S 中最小的没有出现过的自然数。

这里的排列一开始是数列，但是同学们注意到排列的做法并不能直接用到数列上。

2.6　指示器变量

如果 X 形如“满足 xxx 条件的元素的个数”，那么把 X 写成 $\sum_x[x \text{ 满足条件}]$ 常常是有效的。

评注. 这在**期望**中特别常见（称为“指示器随机变量”）。套上期望就得到

$$\mathbb{E}[X] = \sum_x \Pr[x \text{ 满足条件}].$$

例 2.12. $1, \dots, n$ 的所有排列的逆序对数之和是多少？

解答.

$$\begin{aligned}\sum_\pi \operatorname{inv}(\pi) &= \sum_\pi \sum_{i<j} [\pi_i > \pi_j] \\ &= \sum_{i<j} \sum_\pi [\pi_i > \pi_j] \\ &= \sum_{i<j} \frac{n!}{2} \\ &= \frac{n!}{2}\binom{n}{2},\end{aligned}$$

其中第二行到第三行使用了排列的对称性，即 $\pi_i > \pi_j$ 和 $\pi_i < \pi_j$ 的排列个数是一样多的。

排列是最对称的东西。

练习 2.13. $1, \dots, n$ 的所有排列 π 的**严格前缀** max **数量**之和是多少？一个位置 i 是严格前缀 max 当且仅当 $\forall j < i. \pi_i > \pi_j$。

2.7　幂的处理

2.7.1　展开乘积

把 $(\sum_i f(i))^m$ 写成 $\sum_{i_1,i_2,\cdots,i_m} f(i_1)f(i_2)\cdots f(i_m)$ 常常是有用的。例如，我们可以用它证明下面的结论：

例 2.14. 设 $X_1, \cdots, X_n$ 是独立随机变量，则

$$\begin{aligned}\mathbb{E}\left[\left(\sum_i X_i\right)^2\right] &= \mathbb{E}\left[\sum_{i,j} X_i X_j\right] \\ &= \sum_{i,j} \mathbb{E}[X_i X_j] \\ &= \sum_{i\neq j} \mathbb{E}[X_i]\mathbb{E}[X_j] + \sum_{i=j} \mathbb{E}[X_i X_j] \\ &= \sum_i \mathbb{E}[X_i^2].\end{aligned}$$

对 $Y_i = X_i - \mathbb{E}[X_i]$ 使用这个结果，我们就得到了 $\operatorname{Var}[\sum_i X_i] = \sum_i \operatorname{Var}[X_i]$。

考虑更高阶矩，然后考虑所有阶矩，然后想出矩生成函数，然后得到 Chernoff bound。

另外，如果 $f(i)$ 的值只有 $0,1$，那么我们还有结论

$$\left(\sum_i f(i)\right)^{\underline{m}} = \sum_{i_1 \neq i_2 \neq \cdots \neq i_m} f(i_1)f(i_2)\cdots f(i_m).$$

2.7.2　普通幂和下降幂的互相转换

$\sum_i i^k$ 并不好算，而 $\sum_i i^{\underline{k}}$ 容易计算。如果我们在开始处理问题之前进行一下这样的转换，那么问题常常会变得简单许多。

我们可以在 $O(k^2)$（甚至 $O(k\log^2 k)$）内在普通幂多项式 $\sum_i a_i x^i$ 和下降幂多项式 $\sum_i b_i x^{\underline{i}}$ 之间相互转换。有下面几种做法：

做法一：拉格朗日插值

我们可以通过拉格朗日插值在普通幂多项式系数 $a_0,\ldots,a_n$ 和点值 $f(x_0),\ldots,f(x_n)$ 之间相互转换。如果我们将点值取为 $0,1,\ldots,n$，那么也可以方便地在下降幂多项式系数 $b_0,\ldots,b_n$ 和点值 $f(0),f(1),\ldots,f(n)$ 之间相互转换。具体来说，如果我们知道 $b_0,\ldots,b_n$，那么可以通过维护 $\sum_{i\geq k} b_i(x-k)^{\underline{i-k}}$ 来 $O(n)$ 计算所有点值。反过来，假设已知了所有 $f(0),\ldots,f(n)$。注意到

$$f(t) = \sum_i b_i t^{\underline{i}} = t!b_t + \sum_{i>t} b_i t^{\underline{i}},$$

所以可以从高到低递推所有 b_t。这样，我们就通过点值表示的桥梁实现了普通幂与下降幂之间的互转。

做法二：直接递推

可以在 $O(k)$ 内根据 $x^{\underline{k-1}}$ 的普通幂表示来计算 $x^{\underline{k}}$ 的普通幂表示，从而 $O(k^2)$ 计算所有 $x^{\underline{1}},\cdots,x^{\underline{k}}$ 的普通幂表示。反过来，注意到 $i<t$ 的 $a_i x^i$ 和 $b_i x^{\underline{i}}$ 对 t 次及以上的项都没有影响，所以可以类似多项式除法从高到低计算所有 $\sum_i a_i x^i$ 转到 $\sum_i b_i x^{\underline{i}}$ 后的各项系数。具体来说，在 $\sum_i a_i x^i = \sum_i b_i x^{\underline{i}}$ 两侧提取 x^t 的系数得到

$$a_t = [x^t]\sum_i b_i x^{\underline{i}} = b_t + [x^t]\sum_{i>t} b_i x^{\underline{i}}.$$

因此只要维护 $\sum_{i>t} b_i x^{\underline{i}}$ 就可以从高到低计算出所有 b_t。

做法三：斯特林数

我们有结论

$$x^k = \sum_i \left\{ {k \atop i} \right\} x^{\underline{i}}, \quad x^{\underline{k}} = \sum_i \left[{k \atop i} \right] x^i (-1)^{k-i}.$$

然而，即使我们不知道斯特林数的定义，我们也可以通过类似扰动法的方式来计算出斯特林数满足的递推式，从而计算出斯特林数。具体来说，我们有 $x^k = \sum_i \left\{ {k \atop i} \right\} x^{\underline{i}}$，而另一方面有

$$\begin{aligned} x^k = x \cdot x^{k-1} &= \sum_i \left\{ {k-1 \atop i} \right\} x \cdot x^{\underline{i}} = \sum_i \left\{ {k-1 \atop i} \right\} \cdot x^{\underline{i}}(x-i+i) \\ &= \sum_i \left\{ {k-1 \atop i} \right\} \cdot (i x^{\underline{i}} + x^{\underline{i+1}}) \\ &= \sum_i \left(\left\{ {k-1 \atop i-1} \right\} + i \left\{ {k-1 \atop i} \right\} \right) x^{\underline{i}}. \end{aligned}$$

比较系数就得到 $\left\{ {k \atop i} \right\} = \left\{ {k-1 \atop i-1} \right\} + i \left\{ {k-1 \atop i} \right\}$。$\left[{k \atop i} \right]$ 也同理，递推式为 $\left[{k \atop i} \right] = \left[{k-1 \atop i-1} \right] + (k-1) \left[{k-1 \atop i} \right]$。

练习 2.15. 数列求和再再放送. 先计算

$$\sum_{i=0}^{n} i^{\overline{k}} a^i,$$

然后得到又一个

$$\sum_{i=1}^{n} i^k a^i.$$

的 $O(k^2)$ 做法。

这还是个很好的练习。

评注. 因为大家都会 $O(k \log k)$ 求一行斯特林数，这是题解区最常见的做法。

评注. 在上面的子问题中，假设 $0 < a < 1$，令 $n = \infty$，然后重新审视你的结果。

2.8　拆括号容斥

某些问题要求我们计数

$$\sum (1 - [P_1])(1 - [P_2]) \cdots (1 - [P_k]),$$

那么就可以拆括号变成

$$\sum_{S \subseteq [k]} (-1)^{|S|} \sum \prod_{i \in S} [P_i],$$

其中 $\sum \prod_i [P_i]$ 往往是容易算的。相比用集合表述的容斥公式，这种表述会更容易理解和记忆。

全都记不住。

练习 2.16. 通过将 $|A|$ 写成 $\sum_x [x \in A]$，证明集合表述的容斥公式

$$\left| \bigcap_{i=1}^{k} \bar{A}_i \right| = \sum_{S \subseteq [k]} (-1)^{|S|} \left| \bigcap_{i \in S} A_i \right|.$$

很多时候 $\sum\prod_{i\in S}[P_i]$ 只和 $|S|$ 有关。记这个和为 $N(|S|)$，此时有

$$\begin{aligned}&\sum_{S\subseteq[k]}(-1)^{|S|}\sum\prod_{i\in S}[P_i]\\&=\sum_{S\subseteq[k]}(-1)^{|S|}N(|S|)\\&=\sum_i\binom{k}{i}(-1)^iN(i).\end{aligned}$$

更一般地，$\sum\prod_{i\in S}[P_i]$ 可能只和 S 的某些更简单特征有关，比如 $V(S)=\sum_{i\in S}a_i$。记这个和为 $N(V(S))$，可以枚举 $v=V(S)$ 得到

$$\begin{aligned}&\sum_{S\subseteq[k]}(-1)^{|S|}\sum\prod_{i\in S}[P_i]\\&=\sum_{S\subseteq[k]}(-1)^{|S|}N(V(S))\\&=\sum_vN(v)\sum_{S\subseteq[k]}[V(S)=v](-1)^{|S|}.\end{aligned}$$

可以用其他方法（如背包）统计后面这个求和。

我们可以用这个技巧来证明常用情形下的**二项式反演**。记 $g_i=\sum_{|S|=i}\sum\prod_{i\in S}[P_i]$ 为“钦定了 i 个性质的方案数”，那么就有

$$\begin{aligned}f_k&=\text{恰好满足 }k\text{ 个性质的方案数}=\sum_{|S|=k}\sum\prod_{i\in S}[P_i]\prod_{j\notin S}(1-[P_j])\\&=\sum_{|S|=k}\sum\left(\prod_{i\in S}[P_i]\right)\sum_{T\subseteq\bar{S}}(-1)^{|T|}\prod_{j\in T}[P_j]\\&=\sum_{|S|=k}\sum_{T'\supseteq S}(-1)^{|T'|-|S|}\sum\prod_{i\in T'}[P_i]\qquad(T'=S\uplus T)\\&=\sum_n\left(\sum_{|T'|=n}\sum\prod_{i\in T'}[P_i]\right)\left(\sum_{S\subseteq T',|S|=k}(-1)^{|T'|-|S|}\right)\\&=\sum_ng_n\binom{n}{k}(-1)^{n-k}\end{aligned}$$

练习 2.17. 反过来证明

$$g_k=\sum_nf_n\binom{n}{k}.$$

练习 2.18. 另一个方向. 设 $h_i=\sum_{|S|=i}\sum\prod_{i\notin S}[\overline{P_i}]$ 为“至多满足 i 个性质的方案数”，找出 h 和 f 之间的关系。

2.9　综合练习

现在我们来看一个综合练习，它巩固了我们学过的技巧。

例 2.19. 长为 n 的序列，每个都是 1 到 m 中的数。一个序列的价值为不同数的数量的 k 次方，求所有序列的价值和。
你可以认为 k 非常小或者干脆就是个常数。

虽然我们实际上至少可以做到 $O(k(\log k + \log n))$

我们先看看小的情况。对于 $k = 1$，这就是下面的子问题：

例 2.20. 长为 n 的序列，每个都是 1 到 m 中的数。一个序列的价值为其中不同数的数量，求所有序列的价值和。

解答.

$$\begin{aligned}\sum_a w(a) &= \sum_a \sum_{i=1}^{m} [i \text{ 在} a \text{ 中出现}] \\ &= \sum_{i=1}^{m} \sum_a [i \text{ 在 } a \text{ 中出现}] \\ &= \sum_{i=1}^{m} \sum_a (1 - [i \text{ 不在 } a \text{ 中出现}]) \\ &= \sum_{i=1}^{m} (m^n - (m-1)^n) \\ &= m(m^n - (m-1)^n)\end{aligned}$$

另解.

这是同学给出的做法。

$$\begin{aligned}\sum_a w(a) &= \sum_a \sum_{i=1}^{n} [a_i \text{ 这个值在 } a \text{ 中首次出现}] \\ &= \sum_a \sum_{i=1}^{n} [a_i \text{ 这个值在 } \{a_1, \ldots, a_{i-1}\} \text{ 中没有出现}] \\ &= \sum_{i=1}^{n} \sum_{x=1}^{m} \sum_a [a_i = x][x \text{ 在 } \{a_1, \ldots, a_{i-1}\} \text{ 中没有出现}] \\ &= \sum_{i=1}^{n} \sum_{x=1}^{m} (m-1)^{i-1} m^{n-i} = \sum_{i=1}^{n} (m-1)^{i-1} m^{n-i+1} \\ &= m^n \frac{1 - ((m-1)/m)^n}{1 - (m-1)/m} = m(m^n - (m-1)^n).\end{aligned}$$

对于一般的情况，我们首先使用前面提到的幂重写技巧：

$$\begin{aligned}\sum_a w(a)^k &= \sum_a \sum_{1 \le i_1, i_2, \ldots, i_k \le m} [i_1, \ldots, i_k \text{ 都在 } a \text{ 中出现}] \\ &= \sum_{1 \le i_1, i_2, \ldots, i_k \le m} \sum_a [i_1, \ldots, i_k \text{ 都在 } a \text{ 中出现}].\end{aligned}$$

观察小的情况，可以发现

$$\sum_a [i_1, \ldots, i_k \text{ 都在 } a \text{ 中出现}]$$

只与 $i_1,\dots,i_k$ 中不同值的个数相关。并且，假设恰有 t 个不同值，那么由对称性有

$$\sum_a [i_1,\dots,i_k \text{ 都在 } a \text{ 中出现}] = \sum_a [1,\dots,t \text{ 都在 } a \text{ 中出现}].$$

枚举 $i_1,\dots,i_k$ 中不同值的个数，我们就得到

$$\begin{aligned}
&\sum_{1\le i_1,i_2,\dots,i_k\le m} \sum_a [i_1,\dots,i_k \text{ 都在 } a \text{ 中出现}] \\
&= \sum_t \sum_{1\le i_1,i_2,\dots,i_k\le m} [i_1,\dots,i_k \text{ 恰有 } t \text{ 个不同值}] \sum_a [i_1,\dots,i_k \text{ 都在 } a \text{ 中出现}] \\
&= \sum_t \left(\sum_{1\le i_1,i_2,\dots,i_k\le m} [i_1,\dots,i_k \text{ 恰有 } t \text{ 个不同值}] \right) \left(\sum_a [1,\dots,t \text{ 都在 } a \text{ 中出现}] \right)
\end{aligned}$$

现在，两个括号可以分别考虑。前面这部分是

$$\sum_{1\le x_1<x_2<\cdots<x_t\le m} \sum_{1\le i_1,i_2,\dots,i_k\le m} [\{i_1,\dots,i_k\} \text{ 这个集合等于} \{x_1,\dots,x_t\}].$$

由对称性，

$$\sum_{1\le i_1,i_2,\dots,i_k\le m} [\{i_1,\dots,i_k\} = \{x_1,\dots,x_t\}] = \sum_{1\le i_1,i_2,\dots,i_k\le m} [\{i_1,\dots,i_k\} = \{1,\dots,t\}].$$

接下来

$$\begin{aligned}
&\sum_{1\le x_1<x_2<\cdots<x_t\le m} \sum_{1\le i_1,i_2,\dots,i_k\le m} [\{i_1,\dots,i_k\} \text{ 这个集合等于} \{1,\dots,t\}] \\
&= \binom{m}{t} \sum_{1\le i_1,i_2,\dots,i_k\le t} [1,\dots,t \text{ 都在} (i_1,\dots,i_k) \text{ 中出现}].
\end{aligned}$$

注意到这和后半部分是相同形式的问题，所以我们来考虑下面的子问题：

例 2.21. 长为 n 的序列，每个都是 1 到 m 中的数。要求 $1,\cdots,k$ 必须在序列中出现，求方案数。

记

$$f(n,m,k) = \sum_{1\le i_1,\dots,i_n\le m} [1,\dots,k \text{ 都在} (i_1,\dots,i_n) \text{ 中出现}].$$

为这个问题的答案。容斥得到

$$\begin{aligned}
f(n,m,k) &= \sum_{1\le i_1,\dots,i_n\le m} \prod_{j=1}^{k} (1 - [j \text{ 在} \{i_1,\dots,i_n\} \text{ 中不出现}]) \\
&= \sum_{1\le i_1,\dots,i_n\le m} \sum_{S\subseteq\{1,\dots,k\}} (-1)^{|S|} [\forall x\in S,\ x \text{ 在} \{i_1,\dots,i_n\} \text{ 中不出现}] \\
&= \sum_{S\subseteq\{1,\dots,k\}} (-1)^{|S|} \sum_{1\le i_1,\dots,i_n\le m} [\forall x\in S,\ x \text{ 在} \{i_1,\dots,i_n\} \text{ 中不出现}] \\
&= \sum_{S\subseteq\{1,\dots,k\}} (-1)^{|S|} (m-|S|)^n \\
&= \sum_t \binom{k}{t} (-1)^t (m-t)^n
\end{aligned}$$

现在

$$\sum_t \left(\sum_{1\le i_1,i_2,\dots,i_k\le m} [i_1,\dots,i_k \text{ 恰有 } t \text{ 个不同值}] \right) \left(\sum_a [1,\dots,t \text{ 都在 } a \text{ 中出现}] \right)$$
$$= \sum_{t=0}^{k} f(k,t,t) f(n,m,t)$$

而

$$f(n,m,k) = \sum_t \binom{k}{t} (-1)^t (m-t)^n$$

可以 $O(k)$ 计算。故本题可以 $O(k^2)$ 完成。实际上，通过卷积也可以 $O(k \log k)$ 完成。

我们还要说这句话多少遍？

评注. 事实上，$f(n,k,k)$ 就是第二类斯特林数 $k!\left\{ {n \atop k} \right\}$：把 $1,\dots,k$ 视作有标号盒子，$i_1,\dots,i_n$ 视作球即可。

现在来看看另一个推导的可能性。如果在一开始就枚举序列的价值，就得到

$$\begin{aligned}\sum_w w^k \sum_a [w(a)=w] &= \sum_w w^k \sum_a [a \text{ 中恰有 } w \text{ 个不同值}] \\ &= \sum_w w^k \binom{m}{w} \sum_a [a \text{ 中出现的元素恰好是 } \{1,\dots,w\}] \\ &= \sum_w w^k \binom{m}{w} f(n,w,w)\end{aligned}$$

然而这个并不能直接用于计算，因为 w 有意义的范围高达 $[0,\min(n,m)]$。

化简这个式子来用于计算需要比较高超的技巧。

最后我们留下两个练习来巩固一下这个例子中用到的技巧。

练习 2.22. 长为 n 的序列，每个都是 1 到 m 中的数。定义一个序列 a 的权值 $w(a)$ 为 a 中不同数的数量。在 $O(k^2)$ 时间内完成

- 求所有序列的 $(w(a))^{\underline{k}}$ 的和
- 给定 k 次多项式 f，求所有序列的 $f(w(a))$ 的和

2.10　补充的 OI 题及解答

补充的 OI 练习题：洛谷 P5686 P4948 P4091 U77198

例 2.23. [**CSP-S 2019 江西.** 和积和]

$$\sum_{l=1}^{n} \sum_{r=1}^{n} \sum_{i=l}^{r} a_i \sum_{i=l}^{r} b_i.$$

需要 $O(n^2)$.

没有人不会这个题，所以我们就不讲了。

例 2.24. 三角形. 求边长为整数、周长为 n 的三角形的数量，旋转和翻转视为同一种方案（也就是边无序）。需要 $O(1)$。

直接的推法 1

$$\begin{aligned}
&\sum_{i,j,k}[1 \le i \le j \le k \le n][i+j+k=n][i+j>k] \\
&= \sum_{i,j}[1 \le i \le j \le n-i-j][i+j > n-i-j] \\
&= \sum_{1 \le i \le j}[i \le n-2j][i > \frac{n}{2} - j] \\
&= \sum_{j \ge 1}\sum_{i \ge 1}[\frac{n}{2} - j < i \le \min(j, n-2j)]
\end{aligned}$$

接下来分类讨论 $\min(j, n-2j)$ 是哪个。当 $j \le n-2j$ 也就是 $j \le n/3$ 时是 j，否则也就是 $j > n/3$ 时是 $n-2j$。

$j \le n/3$ 的部分如下：

$$\sum_{1 \le j \le n/3}\sum_{i \ge 1}[\frac{n}{2} - j < i \le j].$$

在 i 的合法区间非空，也就是 $j < n/4$ 时，合法的 i 有 $j - \lfloor n/2 - j \rfloor = 2j - \lfloor n/2 \rfloor$ 个。所以这部分是

$$\sum_{n/4 < j \le n/3}(2j - \lfloor n/2 \rfloor),$$

等差数列求和。

$j > n/3$ 的部分如下：

$$\sum_{n/3 < j}\sum_{i \ge 1}[\frac{n}{2} - j < i \le n-2j].$$

在 i 的合法区间非空，也就是 $j < n/2$ 时，合法的 i 有 $n - 2j - \lfloor n/2 - j \rfloor = \lceil n/2 \rceil - j$ 个。所以这部分是

$$\sum_{n/3 < j < n/2}(\lceil n/2 \rceil - j),$$

同样等差数列求和。

直接的推法 2

在最开始的条件转化中，除了先对 j 再对 i 求和外，也可能会变成先对 i 求和再对 j 求和。注意到这样子势必会引入 $j \le \frac{n-i}{2}$ 这样的条件，从而内层对 j 的求和会带上 $\lfloor (n-i)/2 \rfloor$ 这样的东西。回顾我们的第一条化简准则——保持求和的条件简单，这样的转化不应该首先被考虑。首先考虑的应该是计算不等式中不带系数的 i 的合法区间。

不过，即使出现了 $\lfloor i/2 \rfloor$ 这样的求和也不要担心，我们可以将其写成 $\frac{i-(i \bmod 2)}{2}$ 的形式，而 $i \bmod 2 = [i \text{ 是奇数}]$。所以在计算 $\sum_i T(i)(i \bmod 2)$ 时，可以做代换 $i = 2k+1$ 变成 $\sum_{2k+1} T(2k+1)$。

我们将在第三章见到更多更一般的处理带有取整的求和的方法。

容斥的推法

使用容斥可以得到简单一些的推法。

$$\begin{aligned}
&\sum_{i,j,k}[1\le i\le j\le k\le n][i+j+k=n][i+j>k]\\
&=\sum_{i,j,k}[i+j+k=n][1\le i\le j\le n-i-j\le n](1-[i+j\le n-i-j])\\
&=\sum_{1\le i\le j}[j\le n-i-j]-\sum_{1\le i\le j}[j\le n-i-j][i+j\le n-i-j]
\end{aligned}$$

前半部分是

$$\begin{aligned}
&\sum_{1\le i\le j}[j\le n-i-j]\\
&=\sum_{1\le j}\sum_{i=1}^{j}[i\le n-2j]\\
&=\sum_{1\le j\le n/3}\sum_{i=1}^{j}1+\sum_{n/3<j}\sum_{i=1}^{n-2j}1\\
&=\sum_{1\le j\le\lfloor n/3\rfloor}j+\sum_{j=\lfloor n/3\rfloor+1}^{\lfloor n/2\rfloor}(n-2j),
\end{aligned}$$

等差数列求和。

后半部分是

$$\begin{aligned}
&\sum_{1\le i\le j}[j\le n-i-j][i+j\le n-i-j]=\sum_{1\le i\le j}[i+j\le n-i-j]\\
&=\frac{1}{2}\left(\sum_{1\le i,j}[i+j\le n-i-j]+\sum_{1\le i=j}[i+j\le n-i-j]\right)\\
&=\frac{1}{2}\left(\sum_{1\le i,j}\sum_{d}[i+j=d][i+j\le n/2]+\sum_{1\le i=j}[i+j\le n/2]\right)\\
&=\frac{1}{2}\left(\sum_{d}\sum_{1\le i,j}[i+j=d][d\le n/2]+\sum_{1\le i}[i\le n/4]\right)\\
&=\frac{1}{2}\left(\sum_{d=2}^{\lfloor n/2\rfloor}(d-1)+\lfloor n/4\rfloor\right)
\end{aligned}$$

同样是等差数列求和

对称性的推法

首先，注意到 $P(i,j,k)=[i+j+k=n][i,j,k$ 构成三角形$]$ 是完全对称的。如果你知道 Burnside 引理，或者尝试使用对称性并手算系数的话，可以发现

$$\sum_{1\le i\le j\le k}P(i,j,k)=\frac{1}{6}\left(2\sum_{1\le i=j=k}P(i,j,k)+3\sum_{1\le i=j,k}P(i,j,k)+\sum_{1\le i,j,k}P(i,j,k)\right).$$

接下来，注意到

$$\begin{aligned}P(i,j,k) &= [i+j+k=n][i,j,k \text{ 构成三角形}]\\ &= [i+j+k=n][i+j+k>2\max(i,j,k)]\\ &= [i+j+k=n][\max(i,j,k)<\frac{n}{2}]\\ &= [i+j+k=n][i,j,k<\frac{n}{2}]\\ &= [i+j+k=n](1-[i\geq n/2])(1-[j\geq n/2])(1-[k\geq n/2])\end{aligned}$$

注意在 $1\leq i,j,k$ 的限制下，拆括号后如果一项中出现了两个或更多 $[i\geq n/2]$ 的乘积，那么这个项不会有贡献（因为此时 $i+j+k$ 必定超过 n）。类似地，$i=j,k$ 这个求和里面不会拆出来 $[i\geq n/2]$ 的项。所以我们有

$$\sum_{1\leq i=j=k} P(i,j,k) = \sum_{1\leq i}[3i=n] = [3\mid n],$$

$$\begin{aligned}\sum_{1\leq i=j,k} P(i,j,k) &= \sum_{1\leq i=j,k}[i+j+k=n](1-[k\geq n/2])\\ &= \sum_{1\leq i}[n-2i\geq 1]-\sum_{1\leq i}[n/2\leq n-2i]\\ &= \left\lfloor\frac{n-1}{2}\right\rfloor-\left\lfloor\frac{n}{4}\right\rfloor.\end{aligned}$$

以及

$$\begin{aligned}&\sum_{1\leq i,j,k} P(i,j,k)\\ &= \sum_{1\leq i,j,k}[i+j+k=n](1-[i\geq n/2]-[j\geq n/2]-[k\geq n/2])\\ &= \sum_{1\leq i,j,k}[i+j+k=n]-3\sum_{1\leq i,j,k}[i+j+k=n][i\geq n/2]\\ &= \sum_{1\leq i,j,k}[i+j+k=n]-3\sum_{1\leq (i-\lceil n/2\rceil+1),j,k}[(i-\lceil n/2\rceil+1)+j+k=n-(\lceil n/2\rceil-1)]\\ &= \binom{n-1}{2}-3\binom{\lfloor n/2\rfloor-2}{2},\end{aligned}$$

这个做法可以比较方便地从三角形扩展到一般的 k 边形（在旋转和翻转下数等价类）。

本章总结

- 熟悉求和符号和艾弗森括号的使用
- 处理和式的基本方法：简化条件、提出无关项、拆开 $\sum(a_i+b_i)$、下标变换、交换求和号
- 处理和式的常见技巧：扰动法、对称、差分再求和、分部求和法、指示器变量、和的幂的展开
- 拆括号容斥
- 下降幂的差分与求和，普通幂和下降幂的互转

3 取整函数

本章提要

- 同余等差数列的结构
- 处理取整的一些技巧
- 整除分块
- 单位根反演

3.1 课前任务

阅读《具体数学》第三章。本章中最重要的内容包括：底和顶（上/下取整）的定义与等价不等式、含有取整的求和的处理方法、mod 的部分应用。

挑选的部分习题：1 3 7 9 12 16 20 22 31 36 39

3.2 处理取整的一些技巧

技巧一：$\left\lfloor \frac{n}{m} \right\rfloor = \frac{n}{m} - \frac{n \bmod m}{m}$。另外，在分子比较复杂时，提前分离出整除的部分常常是方便的，如

$$\begin{aligned}
\sum_{i=0}^{m-1} \left\lfloor \frac{n+i}{m} \right\rfloor &= \sum_{i=0}^{m-1} \left\lfloor \frac{m\lfloor n/m \rfloor + (n \bmod m) + i}{m} \right\rfloor \\
&= \sum_{i=0}^{m-1} \left\lfloor \frac{n}{m} \right\rfloor + \left\lfloor \frac{(n \bmod m) + i}{m} \right\rfloor \\
&= m \left\lfloor \frac{n}{m} \right\rfloor + \sum_{i=0}^{m-1} [(n \bmod m) + i \geq m] \\
&= m \left\lfloor \frac{n}{m} \right\rfloor + (n \bmod m) \\
&= n
\end{aligned}$$

技巧二：在处理含有取整的求和时，可以引入一个新的求和变量替代取整，然后将原本的取整变为一个不含取整的条件括号。形式化地说，就是

$$\lfloor x \rfloor = \sum_k k[\lfloor x \rfloor = k] = \sum_k k[k \leq x < k+1].$$

技巧三：把一个量写成差分的前缀和再次发挥作用。例如 $\lfloor x \rfloor^2 = \sum_{i \geq 1} (i^2 - (i-1)^2)[i \leq \lfloor x \rfloor] = \sum_{i \geq 1} (i^2 - (i-1)^2)[i \leq x]$。在使用技巧二和三时，要记得交换求和总是很有用。

技巧四：另一种差分技巧，将 $S(N)$ 写作 $S(0) + \sum_{n=1}^{N} (S(n) - S(n-1))$。一般配合 $\lfloor \frac{n}{i} \rfloor - \lfloor \frac{n-1}{i} \rfloor = [i|n]$ 使用。

例 3.1.

$$\begin{aligned}\sum_{i\geq 1}\left\lfloor\frac{N}{i}\right\rfloor &= \sum_{n=1}^{N}\sum_{i\geq 1}\left\lfloor\frac{n}{i}\right\rfloor-\left\lfloor\frac{n-1}{i}\right\rfloor\\ &= \sum_{n=1}^{N}\sum_{i\geq 1}[i|n]\\ &= \sum_{n=1}^{N} d(n),\end{aligned}$$

其中 $d(n)$ 为 n 的约数个数。

例 3.2. 对所有 $1\leq n\leq N$ 求

$$S(n)=\sum_{i=1}^{n} a_i\left\lfloor\frac{n}{i}\right\rfloor.$$

$N\leq 10^6$

解答.

$$\begin{aligned}\nabla S(n) &= S(n)-S(n-1)\\ &= \sum_{i\geq 1} a_i\left(\left\lfloor\frac{n}{i}\right\rfloor-\left\lfloor\frac{n-1}{i}\right\rfloor\right)\\ &= \sum_{i\geq 1} a_i[i|n]\\ &= \sum_{i|n} a_i.\end{aligned}$$

枚举 i，对所有 i 的倍数 n 令 $\nabla S(n)\leftarrow\nabla S(n)+a_i$。最后再做一次前缀和还原出 S。复杂度 $O(nH_n)=O(n\ln n)$。

评注. $\nabla S(n)=A*\mathbf{1}$ 实际上是狄利克雷前缀和，可以 $O(n\log\log n)$ 计算。

另解. 枚举 i，则 a_i 在一个 $\lfloor n/i\rfloor$ 的等值段上都会对 $S(n)$ 造成相同的贡献 $a_i\lfloor n/i\rfloor$。等值段为所有的 $[ki,(k+1)i)$，有 $O(n/i)$ 段。所以问题变成 $O(nH_n)$ 个区间加，最后求所有值，差分即可。

2025 年大家发现了一般序列狄利克雷卷积的 $O(n(\log\log n)^2)$ 算法。参见EI博客。

3.3　整除分块

$\lfloor n/i\rfloor$ 只有 $\leq 2\sqrt{n}$ 种不同的取值（要么 $n/i\leq\sqrt{n}$，要么 $i\leq\sqrt{n}$）。在一个 $\lfloor n/i\rfloor$ 的等值区间内，可以做一些批量的操作（如区间加/求和等）。

如何得知一个等值段的端点？假设目前在 i，我们希望知道最大的满足 $\lfloor n/j\rfloor=\lfloor n/i\rfloor$ 的 j。记 $t=\lfloor n/i\rfloor$，则 $\lfloor n/j\rfloor\geq t\iff n/j\geq t\implies j\leq n/t\implies j\leq\lfloor n/t\rfloor$。所以最大的满足 $\lfloor n/i\rfloor=\lfloor n/j\rfloor$ 的 j 即为 $\lfloor n/t\rfloor$。

例 3.3. 最常见的形式.

$$\sum_{i=1}^{n} f(\lfloor n/i \rfloor) g(i).$$

解答. 根据 $\lfloor n/i \rfloor$ 的值划分等值段，每段内的求和都形如 $\sum_{i=L}^{R} f(t)g(i) = f(t)\sum_{i=L}^{R} g(i)$，那么整个式子可以在 $O(\sqrt{n} \cdot \text{time(计算一次 } g \text{ 的区间和)})$ 内完成。g 的区间和通常可以快速计算（比如提前预处理前缀和）。

评注. 典型的无需预处理也可以快速计算的 g 是 $P(i)r^i$（其中 P 是低次多项式）。

例 3.4. 模积和. 求

$$\sum_{i=1}^{n}\sum_{j=1}^{m} [i \neq j](n \bmod i)(m \bmod j).$$

$n, m \leq 10^9$

解答. 不妨设 $n \leq m$。首先容斥掉 $i \neq j$ 的限制得到

$$\left(\sum_{i=1}^{n} n \bmod i\right)\left(\sum_{j=1}^{m} m \bmod j\right) - \sum_{i=1}^{n} (n \bmod i)(m \bmod j).$$

先看前半部分。乘积的两部分是同一种形式，考虑如何计算其中一个。把 $n \bmod i$ 写作 $n - i\lfloor n/i \rfloor$，则每个 $\lfloor n/i \rfloor$ 的等值段内的 $\sum_i n - i\lfloor n/i \rfloor$ 是容易计算的。

再看后半部分，类似地改写 mod 得到

$$\sum_{i=1}^{n} (n - i\lfloor n/i \rfloor)(m - i\lfloor m/i \rfloor).$$

扩展前面等值段的想法，考虑 i 的段划分，满足在每一段上 $\lfloor n/i \rfloor$ 相同、$\lfloor m/i \rfloor$ 也相同。注意这些段的断点是由「$\lfloor n/i \rfloor$ 的段划分断点 $\cup$ $\lfloor m/i \rfloor$ 的段划分断点」形成的，所以总共的段数不超过 $2(\sqrt{n} + \sqrt{m})$。

实现时，只要每次在 $\lfloor n/i \rfloor$ 的下一个变值点和 $\lfloor m/i \rfloor$ 的下一个变值点中取一个较小的跳过去即可。

例 3.5. 求

$$\sum_{i \geq 1} \left\lfloor \frac{n}{i^2} \right\rfloor.$$

$n \leq 10^{18}$

解答. 注意到要么 $i \leq \sqrt[3]{n}$，要不然 $i > \sqrt[3]{n} \Rightarrow n/i^2 < \sqrt[3]{n}$，所以 $\lfloor n/i^2 \rfloor$ 只有不超过 $2\sqrt[3]{n}$ 个不同的值。

下一个变值点可以这样算：$\lfloor n/i^2 \rfloor \geq t \iff n/i^2 \geq t \implies i \leq \sqrt{n/t} \implies i \leq \lfloor \sqrt{n/t} \rfloor$. 所以最大的满足 $\lfloor n/i^2 \rfloor = \lfloor n/j^2 \rfloor$ 的 j 即为 $\lfloor \sqrt{n/t} \rfloor$。

例 3.6. 整除分块的嵌套. 求

$$\sum_{i,j\geq 1}\left\lfloor\frac{n}{ij}\right\rfloor.$$

$n\leq 10^{10}$

另外，你能证明这是一个积性函数前缀和吗？

评注. 不难用差分技巧发现原式即为

$$\sum_{i=1}^{N}\sum_{j|i}\sum_{k|\frac{i}{j}}1=\sum_{i=1}^{N}(1*1*1)(i),$$

这是一个积性函数前缀和，可以用 *zky* 筛做到 $\tilde{O}$[1]$(\sqrt{n})$。然而没什么实用价值。

解答.

$$\sum_{i,j\geq 1}\left\lfloor\frac{n}{ij}\right\rfloor=\sum_{i\geq 1}\sum_{j\geq 1}\left\lfloor\frac{\lfloor n/i\rfloor}{j}\right\rfloor.$$

记 $f(n)=\sum_{j\geq 1}\lfloor n/j\rfloor$，则原式即 $\sum_{i\geq 1}f(\lfloor n/i\rfloor)$.

枚举所有 $\lfloor n/i\rfloor$ 的等值段，每次花费 $O(\sqrt{n/i})$ 的复杂度。总复杂度为

$$O\left(\sum_{i=1}^{\sqrt{n}}\sqrt{i}+\sum_{i=1}^{\sqrt{n}}\sqrt{n/i}\right).$$

这个如何计算？使用一些微积分知识，我们有下面的结果

引理 3.7. 对 $\alpha\in\mathbb{R}$，当 $n\to\infty$ 时，

$$\sum_{i=1}^{n}i^{\alpha}\sim\begin{cases}\frac{n^{\alpha+1}}{\alpha+1} & \alpha>-1\\ \ln n & \alpha=-1\\ \zeta(-\alpha)=O(1) & \alpha<-1\end{cases}.$$

另外，在 $\alpha<-1$ 的时候还有

$$\sum_{i\geq n}i^{\alpha}\sim\frac{n^{\alpha+1}}{\alpha+1}.$$

我更愿意称之为「事实」而非「引理」

所以这个复杂度就是

$$O\left(\sum_{i=1}^{\sqrt{n}}i^{1/2}+\sqrt{n}\sum_{i=1}^{\sqrt{n}}i^{-1/2}\right)=O((\sqrt{n})^{3/2}+\sqrt{n}\cdot(\sqrt{n})^{1/2})=O(n^{3/4}).$$

更进一步. 我们可以通过预处理做到更好的复杂度。在先前的例子中，我们已经证明了 $f(n)$ 是约数个数函数 $d(n)$ 的前缀和。对于任何积性函数，我们可以用线性筛筛出其前 n 项，进而求出其前缀和。所以，我们可以用 $O(B)$ 的复杂度预处理所有 $k\leq B$ 的 $f(k)$。对于 $k>B$

[1] $\tilde{O}(f(n))$ 代表 $O(f(n)\log^{O(1)}f(n))$，用在不关心 log 因子的时候。然而，OI 中 log 因子几乎总是重要的。

的部分，我们还是用整除分块计算。假设 $B > \sqrt{n}$。现在，只有 $\lfloor n/i \rfloor \geq B$ 也就是 $i \leq n/B$ 的部分需要使用整除分块，总复杂度为

$$O\left(B + \sum_{i=1}^{n/B} \sqrt{n} i^{-1/2}\right) = O\left(B + n/B^{1/2}\right).$$

为寻求其最小值，我们可以令 $B = n/B^{1/2}$，这样得到 $B = n^{2/3}$。此时，上面的复杂度变成 $O(n^{2/3})$。

3.4　单位根反演

假设存在某个 ω_n 使得

$$\frac{1}{n}\sum_{j=0}^{n-1}(\omega_n^i)^j = [i \bmod n = 0],$$

这个名字应该是 OI 圈生造的。反演在哪？

则把 $[i \bmod n = 0]$ 用这个和式替代往往是有用的。

练习 3.8. 对于 $[i \bmod n = j]$，导出一个类似的和式

本原单位根就满足这个性质。定义如下：

定义 3.9. 单位根（root of unity）. 任何满足 $\omega_n^n = 1$ 的元素 $\omega_n \in \mathbb{F}$ 都是域 $\mathbb{F}$ 中的一个 n 次**单位根**。如果额外满足 $\forall 0 < i < n, \omega_n^i \neq 1$，那么称为 n 次**本原单位根**。

下面我们说单位根的时候都指的是本原单位根。

练习 3.10. 对于本原单位根 ω_n，证明 $\omega_n, \omega_n^1, \omega_n^2, \cdots, \omega_n^{n-1}, 1$ 为方程 $x^n = 1$ 的 n 个**不同**的根。

评注. 目前，你可以认为域是一个可以做加减乘除的集合。常见的域包括有理数 $\mathbb{Q}$，实数 $\mathbb{R}$，复数 $\mathbb{C}$，以及 *OI* 中最常见的有限域 $\mathbb{F}_p$（可以认为是 $\bmod p$ 下做运算）。需要注意前面几个和最后一个的性质很不相同。

这是因为它们的特征不同

定理 3.11. “单位根反演”. 设 ω_n 是一个 n 次本原单位根，那么

$$\frac{1}{n}\sum_{j=0}^{n-1}(\omega_n^i)^j = [i \bmod n = 0].$$

证明. 考虑

$$\sum_{j=0}^{n-1}(\omega_n^i)^j$$

是什么。首先有 $\omega_n^i = \omega_n^{i \bmod n}$。所以不妨设 $0 \leq i < n$。当 $i = 0$ 时，每一项都是 $\omega_n^{0 \cdot j} = 1$；当 $i \neq 0$ 时，这是等比数列求和

$$\frac{1 - (\omega_n^i)^n}{1 - \omega_n^i} = 0.$$

所以

$$\sum_{j=0}^{n-1}(\omega_n^i)^j=\begin{cases}n & i \bmod n=0\\ 0 & \text{otherwise}\end{cases}=n[i \bmod n=0]$$

□

例 3.12. 在特征不为 2 的域中，±1 是 $x^2=1$ 的两个根，所以 -1 可以作为 ω_2。于是我们有

$$[n \bmod 2=0]=\frac{1^n+(-1)^n}{2},\quad [n \bmod 2=1]=\frac{1^n-(-1)^n}{2},$$

这也许是最常见的应用。

在复数域 $\mathbb{C}$ 上，$e^{2\pi i/n}$ 是一个 n 次单位根。

在数论中有原根的概念。我们只考虑模素数 p 的情况：如果 $g,g^2,\cdots,g^{\varphi(n)-1},g^{p-1}=1$ 在 mod p 下互不相同，则称 g 是 mod p 下的一个原根。换句话说，g 是 mod p 下的一个 $p-1$ 次单位根。例如，3 是 998244353 的一个原根，所以是 mod 998244353 时的 $998244352=2^{23}\times7\times17$ 次单位根。

ω_n^d 是一个 $n/\gcd(n,d)$ 次单位根。所以，在 mod p 时，如果 $n\mid(p-1)$，那么 $\omega_{p-1}^{(p-1)/n}=g^{(p-1)/n}$ 就是一个 n 次单位根，其中 g 是 mod p 下的原根。

例 3.13. 求 $\sum_i\binom{n}{2i}x^{2i}$ 的封闭形式.

2 换成 3 或者 4 呢？

解答.

$$\begin{aligned}\sum_i\binom{n}{2i}x^{2i}&=\sum_i[i \bmod 2=0]\binom{n}{i}x^i\\&=\sum_i\binom{n}{i}x^i\cdot\frac{1^i+(-1)^i}{2}\\&=\frac{(1+x)^n+(1-x)^n}{2}.\end{aligned}$$

更一般地，不难得到

$$\sum_i\binom{n}{ki+r}x^{ki+r}$$

的做法（$0\le r<k$，假设 ω_k 存在）。

评注. 实际上这是**离散傅里叶变换**（*Discrete Fourier Transform, DFT*）的一个特例。离散傅里叶变换 $\mathtt{DFT}_n$ 事实上是在 $\bmod(x^n-1)$ 下做的。换句话说，

$$\mathtt{IDFT}_n(\mathtt{DFT}_n(F(x)))=F(x) \bmod (x^n-1),$$

而 $[i \bmod n=0]$ 刚好就是 $[x^0](x^i \bmod (x^n-1))$，做一下上面的正逆变换刚好就是单位根反演的形式。

3.4.1　扩域

有时候单位根不存在，此时怎么办呢？我们以 ω_3,ω_4 为例。一般情况涉及到比较深的背景知识。

先看 ω_3。我们把原本的数扩充成 $a+b\omega_3$ 的形式（用标准的术语来说，我们在 $\mathbb{F}[\omega_3]$ 中做运算），然后考虑加法和乘法的形式。加法就是对应项相加，而乘法是

$$(a+b\omega_3)(c+d\omega_3)=ac+(ad+bc)\omega_3+bd\omega_3^2.$$

对于这里新出现的 ω_3^2，我们注意到 ω_3 满足 $1+\omega_3+\omega_3^2=0$，所以可以把 ω_3^2 用 $-1-\omega_3$ 替换。

ω_4 也是类似的，只需要注意到 $\omega_4^4-1=(\omega_4-1)(\omega_4+1)(\omega_4^2+1)=0$ 导致 $\omega_4^2+1=0$。

在一般情况下，ω_k 的幂之间的关系由其**最小多项式** $M(x)$ 决定。假设其次数 $\deg M(x)=m$，那么 $1,\omega_k,\ldots,\omega_k^{m-1}$ 之间线性无关，元素之间的乘法可以视为在 $\bmod M(x)$ 下做运算。这部分可以参考域论以及 Galois 理论。

3.5　Kummer 定理

对于素数 p，记 $v_p(n)$ 为 n 的质因子分解中 p 的指数。

定理 3.14. Kummer 定理. $v_p(\binom{a+b}{a})$ 等于 a 和 b 在 p 进制下做加法时的进位次数。

证明.

- $v_p(n!)$ 是多少？
- $v_p(\binom{a+b}{a})$ 是多少
- $\lfloor a/p^k\rfloor+\lfloor b/p^k\rfloor$ 和 $\lfloor(a+b)/p^k\rfloor$ 的关系是什么？

□

3.6　补充的 OI 题及解答

补充的 OI 题：洛谷 P2260 P6826 P5170 P5591

例 3.15. 月下轻花舞. 求

$$\sum_{i=l}^{r}(i-1)\sum_{j=1}^{n}\lceil\log_i j\rceil.$$

解答.

$$\begin{aligned}\sum_{i=l}^{r}(i-1)\sum_{j=1}^{n}\lceil\log_i j\rceil&=\sum_{i=l}^{r}(i-1)\sum_{j=1}^{n}\sum_{k\geq 1}[k\leq\lceil\log_i j\rceil]\\&=\sum_{i=l}^{r}(i-1)\sum_{j=1}^{n}\sum_{k\geq 1}[k<\log_i j+1]\\&=\sum_{i=l}^{r}(i-1)\sum_{j=1}^{n}\sum_{k\geq 1}[i^{k-1}<j]\\&=\sum_{k\geq 1}\sum_{i=l}^{r}(i-1)\max(n-i^{k-1},0)\end{aligned}$$

注意到当 $k-1>\log_2 n$ 时，求和项一定为 0。所以 k 只需要枚举到大概 $\log_2 n$ 左右。对每个 k，考虑 $n-i^{k-1}\geq 0$ 可以得到 i 有意义的范围，然后里面的东西就是一些 $\sum i^k$ 的形式。

我们在前面已经讲过如何对一个固定的 n 在 $O(k^2)$ 时间内计算

$$P_k(n)=\sum_{i=1}^{n} i^k,$$

而现在的问题是对一些不同的 k 和 n_k 计算这个式子。事实上，我们可以在以前的推导过程中将 n 视为**变量**，这样我们就能得到关于 n 的**多项式** $P_k(n)$。这样，我们就可以用 $O(k)$ 的代价查询一个 n_k 的结果。

我们讲过的两种推导都需要 $O(k^3)$ 的预处理时间。而通过生成函数，我们可以得到一个 $O(k^2)$ 的预处理算法。考虑 EGF

虽然我们还没有讲

$$\sum_k \frac{x^k}{k!}\sum_{i=0}^{n-1} i^k=\sum_{i=0}^{n-1} e^{ix}=\frac{e^{nx}-1}{e^x-1}.$$

我们提前预处理伯努利数 $B(x)=\frac{x}{e^x-1}$ 的系数。那么提取 $[x^k]$ 就得到

$$\begin{aligned}[x^k]\frac{e^{nx}-1}{e^x-1}&=[x^k]\left(\frac{e^{nx}-1}{x}\right)\left(\frac{x}{e^x-1}\right)\\&=\sum_{i=0}^{k} B_{k-i}[x^i]\left(\frac{e^{nx}-1}{x}\right)\\&=\sum_{i=0}^{k} B_{k-i}\frac{n^{i+1}}{(i+1)!}.\end{aligned}$$

例 3.16. 类欧几里得算法.

解答. TODO.

例 3.17. 小猪佩奇学数学.

解答. TODO.

本章总结

- 同余等差数列的结构：$((a+it) \bmod n)_{i=0}^{n-1}$ 形成 $d=\gcd(n,t)$ 个相同的长为 n/d 的排列。
- 处理取整的一些技巧：分离整数和小数部分、用新变量和不等式条件替换取整、差分再求和、关于 n 差分
- 整除分块：$\lfloor n/i\rfloor$ 的取值很少，用 $\lfloor n/\lfloor n/i\rfloor\rfloor$ 计算当前取值的 i 的右端点
- $\sum_{i=1}^{n} i^\alpha \sim \begin{cases}\frac{n^{\alpha+1}}{\alpha+1} & \alpha>-1\\ \ln n & \alpha=-1\\ O(1) & \alpha<-1\end{cases}$
- 单位根反演：$[n \bmod k=0]=\frac{1}{k}\sum_{i=0}^{k-1}\omega_k^{in}$

4 数论

本章提要

- 同余等差数列的结构
- 处理取整的一些技巧
- 整除分块
- 单位根反演

4.1 课前任务

4.2 整除

定义 4.1. 对于整数 n 和正整数 m，如果存在整数 k 使得 $n = mk$，我们就称 m 整除 n，记作 $m \mid n$。

评注. 以下几种说法是大致等价的：

- $m \mid n$
- n/m 是整数
- n 是 m 的倍数
- m 是 n 的约数/因数/因子
- $n \bmod m = 0$
- $n \equiv 0 \pmod{m}$

仅有的分歧在于，在 n 或者 m 为负时，人们对于这些说法的接受程度不同。例如，很少有人会对负的 n 使用约数的说法。

性质 4.2.

- $d \mid n, d \mid m \Rightarrow d \mid (an + bm)$
- 传递性 $a \mid b, b \mid c \Rightarrow a \mid c$
- $d \mid n \Leftrightarrow \frac{n}{d} \mid n$

例 4.3. 约数与倍数位置求和. 给定数组 $a_1, a_2, \ldots, a_n$。对每个 i，求

$$b_i = \sum_{d \mid i} a_d, \quad c_i = \sum_{i \mid d} a_d.$$

定义 4.4. $\gcd, \operatorname{lcm}$；互质 $\perp$

性质 4.5.

1. $\gcd(a, b) = \gcd(a, a \pm b) = \gcd(b, a \pm b)$

2. $\gcd(a,b) = \gcd(a, a \bmod b) = \gcd(b, a \bmod b)$
3. $x \mid a, x \mid b \Leftrightarrow x \mid \gcd(a,b)$

算法 4.6.（扩展）Euclid 算法. 给定整数 a,b，Euclid 算法可以计算出 $\gcd(x,y)$。它的扩展可以进一步计算出一组整数 x,y 使得 $ax+by=\gcd(a,b)$。

Euclid 通常翻译成“欧几里得”。

定理 4.7. Bézout 定理. 对于任意整数 a,b,t，设 $d=\gcd(a,b)$，则关于 x,y 的不等方程

$$ax+by=t$$

有解当且仅当 $d \mid t$。假设其一组解为 (x_0,y_0)，则解集为

$$\{(x_0+k\frac{b}{d}, y_0-k\frac{a}{d}) : k \in \mathbb{Z}\}.$$

Bézout 通常翻译成“贝祖”或者“裴蜀”，前者发音比较正确。

4.3　素数

如果一个正整数 $p>1$ 只有 1 和 p 两个因子，则称 p 为素数（prime，也称质数）。如果一个正整数 $n>1$ 不是素数，则称它为合数（composite）。特别地，1 既不是素数也不是合数。

为什么 1 这么特殊？这是一个不太平凡的问题。

如果素数 p 整除 n，则称 p 是 n 的一个素因子/质因子。

事实 4.8. 如果 n 是合数，那么它有一个不超过 $\sqrt{n}$ 的素因子

练习 4.9. 素性测试. 据此得到一个 $O(\sqrt{n})$ 的素数判断算法。

评注. *素性测试是 P 的——AKS 算法可以在 $\tilde{O}(\log^{12} n)$ 复杂度内判断一个数是否是素数。工业上常用的 Miller-Rabin 素性测试可以在 $O(\log(1/\epsilon)\log n)$ 次加法和乘法内以至少 $1-\epsilon$ 的正确率判断一个数是否是素数。*

Lenstra-Pomerance 后来将其改进至 $\tilde{O}(\log^6 n)$

定理 4.10. 素数定理. 令 $\pi(n)$ 表示不超过 n 的素数的个数，则

$$\frac{\pi(n)}{n} \sim \frac{1}{\ln n}.$$

也就是说，大约有 $\frac{1}{\ln n}$ 个整数是素数。

推论 4.11. 设 p_n 为第 n 个素数，则 $p_n \sim n\ln n$。

$$\sum_{\text{素数}p \le n} \frac{1}{p} = \ln\ln n + O(1).$$

算法 4.12. Eratosthenes 筛法. 给定整数 n，埃拉托色尼筛法可以在 $O(n\log\log n)$ 的时间内计算出不超过 n 的所有素数。

Eratosthenes 通常翻译成“埃拉托色尼”。然而这个名字还是很长，所以我们一般说“埃氏筛法”。“筛法”也

算法 4.13. 线性筛/欧拉筛. 给定整数 n，线性筛法可以在 $O(n)$ 的时间内计算出不超过 n 的所有素数。

练习 4.14. 在 $O(n)$ 时间内计算出 $1, 2, \ldots, n$ 中每个数的最小素因子与次数。

算法 4.15. 区间筛. 可以在 $O((r-l+1)\log\log r + \sqrt{r})$ 时间内筛出 $[l, r]$ 内的所有素数。

定理 4.16. 唯一分解定理/算数基本定理. 任何整数 n 都可以分解成

$$n = \prod_p p^{v_p(n)}$$

的形式，其中 p 取遍所有素数。
这种分解被称为素因子分解，这样的分解是唯一的。我们用 $v_p(n)$ 表示 n 的素因子分解中 p 的指数。

评注. 一般来说，我们会忽略 $v_p(n) = 0$ 的项。

为了证明这个定理，我们先给出一个引理。

引理 4.17. Euclid 引理. 如果 p 是素数，且 $p \mid ab$，则 $p \mid a$ 或 $p \mid b$。

证明. 若 $p \mid a$ 则定理成立；否则设 $p \nmid a$。由于 p 是素数，有 $\gcd(a, p) = 1$，故存在整数 x, y 使得 $ax + py = 1$。那么 $b = b(ax + py) = x \cdot ab + yb \cdot p$ 是 p 的倍数。 □

推论 4.18. 如果 p 是素数，且 $p \mid a_1 a_2 \cdots a_n$，则存在 i 使得 $p \mid a_i$。

练习 4.19. 证明唯一分解定理。

练习 4.20. 计算 $n!$ 的素因子分解。

引理 4.21. 整除与素因子分解.

- $d \mid n$ 当且仅当对每个 p 都有 $v_p(d) \le v_p(n)$
- $v_p(\gcd(n, m)) = \min\{v_p(n), v_p(m)\}$
- $v_p(\mathrm{lcm}(n, m)) = \max\{v_p(n), v_p(m)\}$
- $n \perp m$ 当且仅当 n, m 的质因子集合不交
- $k \perp n, k \perp m \Leftrightarrow k \perp nm$
- 若 $d \perp n$，则 $d \mid nm \Longleftrightarrow d \mid m$

4.3.1　积性函数

定义 4.22. 若定义在正整数上的函数 f 满足 $n \perp m \Rightarrow f(nm) = f(n)f(m)$，则称 f 为积性函数。

练习 4.23. 使用线性筛 $O(n)$ 计算任意积性函数。

4.4　取模和同余

如果 $(a-b) \bmod m = 0$，则称 a 与 b 模 m 同余，记作 $a \equiv b \pmod m$。同余是一种**等价关系**。由定义，$a \equiv b \pmod m \iff m \mid (a-b) \iff \exists k, a+km=b$。

性质 4.24. 同余的运算性质.

- 若 $a_1 \equiv a_2 \pmod m$，$b_1 \equiv b_2 \pmod m$，则 $a_1 \pm b_1 \equiv a_2 \pm b_2 \pmod m$、$a_1 b_1 \equiv a_2 b_2 \pmod m$
- $ad \equiv bd \pmod{md} \iff a \equiv b \pmod m$
- 如果 $d \perp m$，则 $ad \equiv bd \pmod m \iff a \equiv b \pmod m$
- $a \equiv b \pmod{cm} \implies a \equiv b \pmod m$

练习 4.25. 证明 $ad \equiv bd \pmod m \iff a \equiv b \pmod{\frac{m}{\gcd(d,m)}}$

练习 4.26. 不要使用 int128 或 long double，实现以下功能：

- 进行一系列加减乘法运算，最后分别输出对 m_1，$m_2, \cdots, m_k$ 取模的结果，保证 $m_1 m_2 \cdots m_k \le 10^9$。只做一遍而不是做 k 遍。
- 进行一系列加减乘法运算，最后输出对 2^{63} 取模的结果。

模 m 同余作为一个等价关系会将整数划分为若干等价类，i 所在的等价类为 $\bar{i} = \{i+km : k \in \mathbb{Z}\}$。对每个等价类选择一个代表元，这些代表元组成的集合被称为模 m 的一个完全剩余系，简称剩余系。通常来说，我们会选择 $\{0, 1, 2, \ldots, m-1\}$ 作为模 m 的剩余系。

评注. 用抽象代数的语言来说，所有等价类构成的集合称为 $\mathbb{Z}/m\mathbb{Z}$，它是一个环。

4.4.1　逆元

如果存在 b 使得 $ab \equiv 1 \pmod m$，则称 b 是 a 在模 m 下的逆或逆元，记作 a^{-1}。

定理 4.27. a 在模 m 下有逆元当且仅当 $\gcd(a, m) = 1$。如果 a 在模 m 下有逆元，则它的逆元是唯一的。

推论 4.28. 若 p 是质数，则任何非零元素在模 p 下都有逆元。

练习 4.29. 利用 Euclid 算法判断逆元存在性，如果存在则求出逆元。

算法 4.30. 递推求逆元. 对于素数 p，可以在 $O(n)$ 时间内递推 $1, 2, \ldots, n$ 在模 p 下的逆元。具体来说，设 $k = \lfloor p/i \rfloor$，$r = p \bmod i$，那么 $p = ki + r$。于是

$$\begin{aligned} ki + r &\equiv 0 \pmod{p} \\ k + ri^{-1} &\equiv 0 \pmod{p} \\ i^{-1} &\equiv -kr^{-1} \pmod{p} \end{aligned}$$

由于 $r < i$，我们可以据此从小到大计算出 i^{-1}。

算法 4.31. 线性求逆元. 可以在 $O(n + \log p)$ 时间内计算出 n 个数在模 p 下的逆元。

定义 4.32. $\{1 \le a < m : \gcd(a, m) = 1\}$ 称为模 m 的一个简化剩余系。其大小被称为 **Euler 函数**，记作 $\varphi(m)$。

Euler 通常被翻译为“欧拉”。欧拉是历史上最伟大的数学家之一。

练习 4.33. 对于素数 p，证明 $\varphi(p^k) = p^k - p^{k-1}$。

定理 4.34. Euler 定理. 如果 $\gcd(a, m) = 1$，则 $a^{\varphi(m)} \equiv 1 \pmod{m}$。

推论 4.35. Fermat 小定理. 如果 p 是素数，且 $p \nmid a$，则 $a^{p-1} \equiv 1 \pmod{p}$。

Fermat 通常被翻译为“费马”或“费尔马”。

推论 4.36. 如果 p 是素数，且 $p \nmid a$，则 $a^{-1} \equiv a^{p-2} \pmod{p}$。

4.4.2　中国剩余定理

定理 4.37. 中国剩余定理. 设 $m_1, m_2, \ldots, m_k$ 两两互素，则同余方程组

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

在模 $m_1 m_2 \cdots m_k$ 下有唯一解。

推论 4.38. 若 $m_1, \ldots, m_k$ 两两互素，则 $\forall i, x \equiv 1 \pmod{m}_i$ 等价于 $x \equiv 1 \pmod{m_1 m_2 \cdots m_k}$

练习 4.39. 证明欧拉函数是积性函数。

中国剩余定理的主要作用是将模合数转为模若干个素数幂下的运算。相比一般的合数，素数幂的性质往往要好得多。

4.4.3　同余等差数列

序列 $(ik \bmod n)_{i=0}^{n-1}$ 长什么样子？当 $\gcd(k,n)=1$ 时，这是 $0,\ldots,n-1$ 的一个排列。这是因为对任何 $0 \le i < j < n$，$n \nmid (j-i)k$，从而 $ik \bmod n \neq jk \bmod n$。

当 $\gcd(k,n)=d$ 时，$ik \bmod n = d(i(k/d) \bmod (n/d))$，而 $\gcd(k/d, n/d)=1$。所以此时序列是 $0,\ldots,n/d-1$ 的一个排列重复 d 次。

练习 4.40. 给定质数 $p \le 10^7$。q 次询问，每次给定 a,k,l,r，计算

$$\left(\prod_{i=l}^{r}(a+ik)\right) \bmod p$$

练习 4.41.

- 把 p 换成**合数** $m \le 10^7$，保证 m 是若干不同质数的乘积。
- * 把 p 换成素数幂 p^k，保证 $p^k \le 10^7$
- * 把 p 换成任意合数 m，保证 $m \le 10^7$
- ** 把 p 换成任意合数 m，保证 $qm \le 10^11$

5 二项式系数和生成函数

本章提要

- 二项式系数及相关恒等式
- 组合计数原理和常见模型
- 生成函数初步

5.1 课前任务

阅读《具体数学》第五章的前四节。本章最重要的内容有以下几点：二项式系数及相关恒等式、大量练习、差分和二项式反演、生成函数初步。

挑选的习题：2 3 4 5 8 14 15 16 35 36 37 40 41 43 45 46 47 59 61 63 64 67 70 71 73 74 76 77 79 这是两周的量

5.2 组合计数原理

5.2.1 加法、乘法、减法原理

- 加法原理：$|A \uplus B| = |A| + |B|$
- 乘法原理：$|A \times B| = |A||B|$
  - 扩展的乘法原理：假设 S 是 $A \times V$ 的一个子集。如果对每个 $x \in A$，都有 $\#\{y : (x, y) \in S\} = m$，则 $|S| = m|A|$
- 减法（容斥）原理：$|A - B| = |A| - |A \cap B|$

5.2.2 商集与除法原理

在很多时候，问题中会显式地或隐式地有“... 视为同一种方案”这种表述。有时，我们可以先无视“视为同一种方案”，然后再除掉一个因子来得到答案。有时，我们则不能这样做。这往往是计数中难以理解的部分。除法原理何时能使用？不能使用除法原理又该怎么办？我们将用等价关系、等价类和商集的概念来说明。

关系和等价关系

“... 视为同一种方案”实际上是在描述**等价关系**。换句话说，我们需要对所有的 x, y 定义“x 和 y 能不能视为同一种东西”。如果可以，我们就记 $x \sim y$。显然，这个定义需要满足

- 自反性 $x \sim x$
- 对称性 $x \sim y \iff y \sim x$
- 传递性：如果 $x \sim y$，$y \sim z$ 那么 $x \sim z$

一般来说，我们可以在集合上定义任意的**二元关系** R，用来描述两个元素 x 和 y 之间是否有某种关系。常见的关系有 $<, \leq, =, |, \subseteq$ 等等。如果 R 满足上面三条性质，那么我们就

称 R 是一个**等价关系**。例如，$=$ 和 $\equiv \pmod m$ 都是等价关系。又比如说，在给一个环染色时，“可以通过旋转变得一样”也是一个等价关系。

可以用有向图来表示一个关系：如果 xRy，那么就连边 $x \to y$。对于满足对称性的关系（如等价关系），自然可以用无向图表示。

等价类和商集

对于集合 S，其上的一个的等价关系 R 会把 S **划分**成若干个**等价类**，满足每个等价类内部的元素两两等价，来自不同等价类的元素两两不等价。在无向图上，这表现为若干个团。而商集 S/R 就定义为以这些等价类为元素构成的集合。也就是说，S/R 是以若干两两不交的 S 的子集为元素构成的集合，每一个元素都是 S 关于 R 的一个等价类。

常见的做法是在每个等价类内选一个**代表元**来表示 S/R 的一个元素。例如，考虑 $\mathbb{Z}$ 上的等价关系 $\equiv \pmod m$，我们用 $\bar{x} = x + m\mathbb{Z}$ 来代表所有 $\equiv x \pmod m$ 的元素形成的等价类。那么商集的元素就是所有的 $\bar{i}$，其中 $0 \le i < m$。一般来说，对于 $x \in S$，我们用 $\bar{x}$ 代表 S/R 中包含 x 的那个等价类。

除法原理

现在，“... 视为同一种方案，求方案数”用我们的语言就可以表示为：在一个等价关系下，求等价类的数量，或者说商集的大小。那么可以使用除法原理的条件也是显然的：如果所有等价类的大小都相同，比如说是 m，那么 $|S/R| = |S|/m$。

如果不相同，那么可以先把等价类按照大小 i 划分，然后考虑所有大小为 i 的等价类的并 S_i。那么 $|S/R| = \sum_i |S_i|/i$。

很多时候等价关系是由**群作用**给出的。此时的常用处理方法是 Burnside 引理。

5.3　经典组合计数模型

我们接下来会给出很多经典的组合计数模型。熟悉这些模型是必要的：第一，任何大的未知问题最后都要分解成小的已知问题；第二，我们必须要在实际的问题中使用我们先前学到的技巧；第三，我们已经练习了太多代数推导，该练习一下组合直观了。

5.3.1　球与盒子

例 5.1. 洛谷 P5824 十二重计数法. 由于本题实际需要 FFT，我们只列式子。n 个球装进 m 个盒子里，模型分别为

I. 球之间互不相同，盒子之间互不相同。
II. 球之间互不相同，盒子之间互不相同，每个盒子至多装一个球。
III. 球之间互不相同，盒子之间互不相同，每个盒子至少装一个球。
IV. 球之间互不相同，盒子全部相同。
V. 球之间互不相同，盒子全部相同，每个盒子至多装一个球。
VI. 球之间互不相同，盒子全部相同，每个盒子至少装一个球。

VII. 球全部相同，盒子之间互不相同。
VIII. 球全部相同，盒子之间互不相同，每个盒子至多装一个球。
IX. 球全部相同，盒子之间互不相同，每个盒子至少装一个球。
X. 球全部相同，盒子全部相同。
XI. 球全部相同，盒子全部相同，每个盒子至多装一个球。
XII. 球全部相同，盒子全部相同，每个盒子至少装一个球。

标号

让我们先讨论一下标号的概念。有区别/可区分/有标号/互不相同都指向相同的概念。一般来说，有标号是最通用和方便的说法。在球与盒子都有标号时，可以逐球考虑/逐盒子考虑来计数。在球有标号、盒子无标号时，盒子的任意排列被视为同一种方案（等价类）。此时，常见的处理方法是**强制**一个顺序（“选代表元”），或者使用**除法原理**。

- 注意到当盒子均非空时，每个等价类的大小都为 $m!$（也就是盒子的不同排列都对应盒子有标号时的不同方案）。所以除法原理给出 $f_{\mathrm{VI}}(n,m)=\frac{1}{m!}f_{\mathrm{III}}(n,m)$。
- 有空盒子的情况则更复杂一些，因为空盒子的不同排列即使在盒子有标号时对应的也是同一种方案。假设有 $m-k$ 个空盒子，那么首先有 $\binom{m}{m-k}$ 种方案排列这些空盒子，非空盒子有 $k!$ 种排法，故总的等价类大小是 $\binom{m}{m-k}k!=m^{\underline{k}}$。这样除法原理就给出 $f_{\mathrm{V}}(n,m)=\frac{f_{\mathrm{II}}(n,m)}{m^{\underline{n}}}$

在球无标号、盒子有标号时，有标号球退化为无标号的“单位”，也就是我们只关心每个盒子里装球的**个数**。

- 此时的一个等价类大小为 $\frac{n!}{n_1!\cdots n_m!}$，其中 n_i 为第 i 个盒子里的球数。
- 因为不同等价类大小不同，除法原理无法在此应用。

在球相同、盒子也相同时，盒子的任意排列被视为同一种方案。此时可以强制按照盒子中装的球数来排序。

- 此时的一个等价类在前一个问题中具有大小 $\frac{m!}{m_0!m_1!m_2!\cdots m_n!}$，其中 m_i 表示装了 i 个球的盒子个数。
- 同样因为不同等价类大小不同，除法原理无法在此应用。

有标号球和有标号盒子

I 和 II. 逐球考虑，容易用乘法原理计数得到 $f_{\mathrm{I}}(n,m)=m^n$ 和 $f_{\mathrm{II}}=m^{\underline{n}}$。

如果逐盒子考虑呢？对于 I，枚举每个盒子里的球数得到

$$\begin{aligned}&\sum_{i_1+i_2+\cdots+i_m=n}\binom{n}{i_1}\binom{n-i_1}{i_2}\binom{n-i_1-i_2}{i_3}\cdots\binom{n-i_1-\cdots-i_{m-1}}{i_m}\\&=\sum_{i_1+i_2+\cdots+i_m=n}\frac{n!}{i_1!i_2!\cdots i_m!}\\&=(1+1+\cdots+1)^n=m^n.\end{aligned}$$

对于 II，类似计算得到

$$\sum_{i_1+i_2+\cdots+i_m=n;i_1,i_2,\ldots,i_m\le 1}n!=\binom{m}{n}n!=m^{\underline{n}}.$$

III. 逐球考虑难以处理限制，不过我们可以先把限制容斥掉。容斥后，如果强制了 i 个空盒子，那么就变成 $f_{\mathrm{I}}(n,m-i)$。所以

$$f_{\mathrm{III}}(n,m)=\sum_i \binom{m}{i}(-1)^i f_{\mathrm{I}}(n,m-i)=\sum_i \binom{m}{i}(-1)^i(m-i)^n.$$

也可以逐盒考虑，这样就是

$$\sum_{i_1+i_2+\cdots+i_m=n;i_1,\ldots,i_m\geq 1}\frac{n!}{i_1!i_2!\cdots i_m!}.$$

仍然需要容斥掉 $i_j\geq 1$ 来计算。

生成函数. 考虑关于 n 的 EGF。在这三种情况下，一个盒子的 EGF 分别为 e^x、$1+x$ 和 e^x-1，所以整个 EGF 分别是 e^{mx}、$(1+x)^m$ 和 $(e^x-1)^m$。

然而，考虑 m 的 GF 则寸步难行。所以，我们总是应该关于“被分配对象”建立生成函数。在球盒模型中，被分配对象是球，而用来分配的容器则是盒子。

有标号球和无标号盒子

我们强制一个顺序：按照每个盒子里装的球的最小标号升序排序（空盒子视为 0）。那么对于 IV，我们可以想象前面有一些空盒子，然后后面是一个 VI。所以

$$f_{\mathrm{IV}}(n,m)=\sum_{i=0}^{m} f_{\mathrm{VI}}(n,i).$$

对于 V，唯一的一种排法就是 $m-n$ 个空盒子后面按顺序排着装 $1,2,\ldots,n$ 的盒子。所以 $f_{\mathrm{V}}(n,m)=[n\leq m]$。这正是我们在上一页中通过对等价类的分析得到的 $f_{\mathrm{V}}(n,m)=\frac{1}{m^n}f_{\mathrm{II}}(n,m)$。

对于 VI，我们在上一页中已经得到 $f_{\mathrm{VI}}=\frac{1}{m!}f_{\mathrm{III}}$。事实上，这就是第二类斯特林数 $\left\{{n\atop m}\right\}$ 的一种定义。代入 III 的结果就得到

$$\left\{{n\atop m}\right\}=\frac{1}{m!}\sum_i \binom{m}{i}(-1)^i(m-i)^n.$$

无标号球和有标号盒子

双射方法. 我们可以通过如下操作建立 VII 与 IX 的双射：对于每个合法的 VII 方案，在每个盒子里放一个球即可得到一个合法的 IX 方案，并且这是一个单射；反之，对于每个合法的 IX 方案，在每个盒子里拿走一个球即可得到一个合法的 VII 方案，而这同样是一个单射。所以我们得到 $f_{\mathrm{VII}}(n,m)=f_{\mathrm{IX}}(n+m,m)$。

对于 IX，我们可以使用称为“隔板法”的双射：在 n 个球之间插 $m-1$ 个隔板，这与 IX 的方案有一一对应。而前者的方案数显然是 $\binom{n-1}{m-1}$，所以 $f_{\mathrm{IX}}(n,m)=\binom{n-1}{m-1}$。这样我们也得到 $f_{\mathrm{VII}}(n,m)=\binom{n+m-1}{m-1}$。（也可以枚举第一个盒子放了几个球，然后使用我们以前学过的方法处理递推式）

对于 VIII，我们可以考虑哪些盒子装了球，这有 $\binom{m}{n}$ 种方案。或者，我们也可以使用上一页中的等价类大小分析来得到 $f_{\mathrm{VIII}}(n,m)=f_{\mathrm{II}}(n,m)/n!$。

生成函数. 在这三种情况下，一个盒子的 OGF 分别为 $\frac{1}{1-x}$、$1+x$ 和 $\frac{x}{1-x}$。所以总的 OGF 分别为 $\frac{1}{(1-x)^m}$、$(1+x)^m$ 和 $\frac{x^m}{(1-x)^m}$。

不难发现生成函数处理盒子有区别的情况是方便的。

我们也可以对于先前的有标号情况的生成函数配合除法原理来得到这里的生成函数。对于一个有标号方案，我们要除掉因子 $\frac{n!}{n_1!\cdots n_m!}$ 来得到无标号的方案数。这个因子可以拆分到盒子上，对每个盒子乘上 $n_i!$ 的因子，在最后再除掉 $n!$。那么每个盒子的 GF 就变成 $1/(1-x)$、$1+x$ 和 $\frac{x}{1-x}$。最后除掉 $n!$ 就是在提取系数时不再乘以 $n!$。

无标号球和无标号盒子

对于 XI，我们可以使用和 V 同样的分析，或者也可以使用前面的等价类大小分析来得到同样的 $f_{\mathrm{XI}} = [n \le m]$。

对于 X 和 XII，我们可以类比 VII 与 IX 来得到 $f_{\mathrm{X}}(n,m) = f_{\mathrm{XII}}(n+m,m)$。

对于 X，这是一个拆分数类型的问题。此类问题的直观理解如下：把每个球画成一个点，把一个盒子里装的球画成一行的点，不同盒子画在不同行并对齐，强制盒子里装的球数从上往下不降。这样得到的点阵被称为 **Ferrers** 图。我们将问题转化为一定限制下的 Ferrers 图计数。

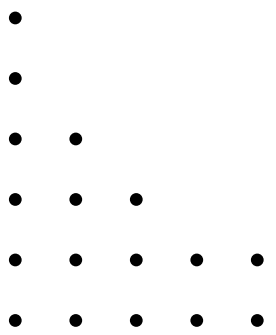


图 5.1: (1, 1, 2, 3, 5, 5) 的 Ferrers 图

从这个图中我们可以注意到其行和列是**对称**的，而列的组合意义是盒子的容量限制。所以，使用不超过 m 个盒子、容量无限制的方案数等于盒子数量无限制、每个盒子容量不超过 m 的方案数。

对 Ferrers 图 DP. 下面省略 f_{X} 的下标 X，即写作 $f(n,m)$。首先，我们可以通过枚举最左边有几列高度为 m，得到

$$f(n,m) = \sum_{i \ge 0} f(n-im, m-1).$$

通过相减抵消求和，可以得到 $f(n,m) = f(n-m,m) + f(n,m-1)$，这是一个 $O(nm)$ 的 dp。或者，我们也可以枚举最左边一列的高度，再通过相减抵消求和得到相同的 dp。

$$f(n,m) = \sum_{i \le m} f(n-i,i) = f(n,m-1) + f(n-m,m)$$

这个 dp 还有另一个理解：考虑行数够不够 m，如果不够那么是 $f(n,m-1)$，如果够那么可以把第一列删掉，这样是 $f(n-m,m)$。总之，如果 $m\le\sqrt{n}$，我们就得到了一个 $O(n\sqrt{n})$ 的做法。

如果 $m>\sqrt{n}$，我们可以想象 Ferrers 图的结构：左边有若干高度 $>\sqrt{n}$ 的列（不超过 $\sqrt{n}$ 个），右边是高度不超过 $\sqrt{n}$ 的列。记 $B=\lfloor\sqrt{n}\rfloor+1$ 为左边的列高度的可能最小值，后面的讨论中都假设 B 是独立于 n 的变量。

如果我们可以在 $O(n^2/B)$ 的时间内计算出所有 $g_B(i)$，表示每列高度都在 $[B,m]$ 内的 i 个点的 Ferrers 图的数量，那么就可以枚举左右的点数计算答案：

$$f(n,m)=\sum_i g_B(n-i)f(i,B-1).$$

而 $f(i,B-1)$ 可以通过上一页的 dp 在 $O(nB)=O(n\sqrt{n})$ 时间内计算。

现在来考虑 g_B 的计算。设 $g(n,k)$ 表示列数恰好为 k 且每列高度都在 $[B,m]$ 内的 Ferrers 图数量，则 $k\le n/B$。我们考虑有没有高度为 B 的列。如果有，那么这样的方案数为 $g(n-B,k-1)$；如果没有（也就是都 $>B$），那么我们可以把最后一行删去，所以方案数是列数恰好为 k 且每列高度都在 $[B,m-1]$ 内的 $n-k$ 个点的 Ferrers 图数量。而这个可以用高度都在 $[B,m]$ 内的方案数减去最高高度恰好为 m 的方案数来计算，其中前者就是 $g(n-k,k)$，而后者可以通过删掉第一列得到 $g(n-k-m,k-1)$。所以

$$g(n,k)=g(n-B,k-1)+g(n-k,k)-g(n-k-m,k-1).$$

再对 k 做一下前缀和就可以计算出 $g_B(n)$。

我们还可以对全部的 n 计算。如果直接卷积，那么需要 $O(n^2)$ 时间。然而，如果把 $f(i,B-1)$ 作为初值来做 g 的 dp，那么就可以在 $O(n\sqrt{n})$ 时间内算出全部的 $f(n,m)$。或者说，现在 $g(n,k)$ 的含义变为：**额外列数**恰好为 k 的 Ferrers 图数量，其中额外列指的是这些高度在 $[B,m]$ 内的列。转移和之前是相同的，只是初值变成了 $f(i,B-1)$。

Ferrers 图的左下正方形. 另一种计算 $f(n,m)$ 的方法是考虑包含左下角的极大正方形。假设这个正方形的边长是 i，那么这个正方形上面的部分就是一个高度不超过 $m-i$、宽度不超过 i 的 Ferrers 图，右边的部分是一个高度不超过 i 的 Ferrers 图，而正方形本身提供了 i^2 个点，所以 $i\le\sqrt{n}$。上面和右边的部分都可以类似前面的 dp 计算。

可能也可以用之前的手法算出所有 n 的结果。

生成函数. 考虑高为 i 的行有几个，不难写出生成函数

$$\prod_{i=1}^{m}\frac{1}{1-x^i}.$$

可以通过 $\ln-\exp$ 做到 $O(n\log n)$，或者使用 q-analog 理论得到若干种不需要 FFT 的 $O(n\sqrt{n})$ 做法。

Burnside 引理　我们也可以回到最开始，通过 Burnside 引理处理“在任意排列下等价”这个限制。枚举所有的置换，此置换的不动点即是说对置换中的每个环，其上的盒子都相同。用 EGF 完成这个枚举：考虑每个环的 GF，然后用 exp 组合。一个环的 EGF 即为 $\sum_{i\geq 1}\frac{y^i}{i}F(x^i)$，其中 $F(x)=1/(1-x)$ 是一个盒子的 GF。那么总的就是

$$\frac{1}{m!}\left[\frac{y^m}{m!}\right]\exp\left(\sum_{i\geq 1}\frac{y^i}{i}\frac{1}{1-x^i}\right)=[y^m]\prod_{j\geq 0}\frac{1}{1-yx^j}$$

或者，我们也可以直接考虑枚举放了 i 个球的盒子有几个，用 y 代表一个盒子的占位元，就得到相同的二元生成函数。

等等，这个生成函数为什么和前面那个长得不一样？证明它们相同并不容易，需要 q-analog 的知识。

5.3.2　变量计数

例 5.2. 对于整数 $x_1,\ldots,x_m\geq 0$，求

- $x_1+x_2+\cdots+x_m=n$ 的方案数
- 在上一条的基础上，加上 $l_i\leq x_i<r_i$ 的限制
- $x_1\leq x_2\leq\cdots\leq x_{m-1}\leq x_m=n$ 的方案数
- 在上一条的基础上，加上 $l_i\leq x_i-x_{i-1}<r_i$ 的限制（设 $x_0=0$）

解答. 第一个问题和之前的 VII 是相同的。

通过换元 $y_i=x_i-x_{i-1}$，可以发现第三（四）个问题和第一（二）个问题是相同的。（事实上，我们以前已经通过和式处理的手法计算过第三个问题）

对于第二个问题，先考虑只有下界 $x_i\geq l_i$ 的情况。通过换元 $y_i=x_i-l_i$，我们可以将其变成第一个问题（n 变成 $n-\sum_i l_i$）。

现在，我们可以不失一般性地假设 $l_i=0$。然后我们可以通过容斥把上界 r_i 变成下界：

$$\begin{aligned}
&\sum_{x_1+x_2+\cdots+x_m=n}\prod_i[x_i<r_i]\\
&=\sum_{x_1+x_2+\cdots+x_m=n}\prod_i(1-[x_i\geq r_i])\\
&=\sum_{x_1+x_2+\cdots+x_m=n}\sum_{S\subseteq[m]}\prod_{i\in S}-[x_i\geq r_i]\\
&=\sum_{S\subseteq[m]}\sum_{x_1+x_2+\cdots+x_m=n}\prod_{i\in S}[x_i\geq r_i]\\
&=\sum_{S\subseteq[m]}(-1)^{|S|}\binom{n-\sum_{i\in S}r_i+m-1}{m-1}
\end{aligned}$$

接下来可以 2^m 枚举 S 来计算。

也可以枚举 $R=\sum_{i\in S} r_i$ 得到

$$\sum_{S\subseteq[m]}(-1)^{|S|}\binom{n-\sum_{i\in S} r_i+m-1}{m-1}$$
$$=\sum_{R}\binom{n-R+m-1}{m-1}\sum_{S\subseteq[m]}(-1)^{|S|}\left[\sum_{i\in S} r_i=R\right],$$

而后者可以背包计算：

$$\begin{aligned}f_{m,R}&=\sum_{S\subseteq[m]}(-1)^{|S|}\left[\sum_{i\in S} r_i=R\right]\\&=\sum_{S\subseteq[m-1]}(-1)^{|S|+1}\left[r_m+\sum_{i\in S} r_i=R\right]+\sum_{S\subseteq[m]}(-1)^{|S|}\left[\sum_{i\in S} r_i=R\right]\\&=-f_{m-1,R-r_m}+f_{m-1,R}.\end{aligned}$$

这样是 $O(nm)$ 的。由于 $n\gg 2^m$ 也是常见的，所以要根据情况选择做法。

评注. 也可以通过生成函数和 $\ln-\exp$ 技巧做到 $O(n\log n)$。

5.3.3　排列

例 5.3. 环排列. n 个数的环排列个数。也就是循环移位下的等价类数量。

解答. 每个环排列通过转圈对应到 n 个不同的排列，所以方案数是 $n!/n=(n-1)!$。

例 5.4. 多重排列. k 种颜色的球，第 i 种颜色的有 n_i 个，求方案数

解答. 每个等价类通过对同色球内部的排列对应到 $\prod_i n_i!$ 个不同的排列，所以方案数是 $\frac{n!}{\prod_i n_i!}$。
或者，也可以一个颜色一个颜色放置，得到

$$\binom{n}{n_1}\binom{n-n_1}{n_2}\binom{n-n_1-n_2}{n_3}\cdots\binom{n-n_1-\cdots-n_{k-1}}{n_k}=\frac{n!}{\prod_i n_i!}.$$

也就是多项式系数 $\binom{n}{n_1,\ldots,n_k}$

例 5.5.（困难）相邻颜色不同的多重排列. P2476 [SCOI2008] 着色方案：同上，但是要求相邻颜色不相同

例 5.6. (困难) P6151 [集训队作业 2019] 青春猪头少年不会梦到兔女郎学姐

例 5.7. P5339 [TJOI2019] 唱、跳、rap 和篮球

5.3.4　数列

例 5.8. 长为 n 的字符集大小为 m 的字符串，不能出现连续 k 个相同字符，求方案数

做法一. 设 $f_{i,j}$ 表示长为 i 的字符串，目前结尾极长连续段长度为 j，且从来没有出现过连续 k 个相同字符的方案数，则 $f_{i,j}=f_{i-1,j-1}(j>1)$ 以及 $f_{i,1}=\sum_{j<k}(m-1)f_{i-1,j}$。$O(nk)$ 直接做或者 $O(k^3\log n)$ 矩阵加速。

做法二. 设 f_i 表示长为 i 的字符串，且从来没有出现过连续 k 个相同字符的方案数。那么首先考虑 mf_{i-1}，然后再减掉不合法的方案数。不合法的情况一定是最后 k 个字符都相同，且倒数第 $k+1$ 个字符和后面的不同。最后 k 个字符有 m 种可能；对于前面的串，由对称性，以每种字符结尾的合法串的个数都是 f_{i-k}/m，所以不合法方案数总共是 $(m-1)f_{i-k}$，故 $f_i=mf_{i-1}-(m-1)f_{i-k}$。$O(n)$ 直接做或者 $O(k^2\log n)$ 甚至 $O(k\log k\log n)$ 线性递推。

做法三. 做法三：容斥，考虑对于每个位置钦定以这个位置结尾有 k 个连续相同字符。然而剩下的事情十分困难，我们只给出生成函数的解释。想象一下钦定之后的序列会长成什么样子：序列被分成若干段，每一段要么是一个未被钦定的字符，要么是因为若干个间距小于 k 的钦定位置而连成的一个大同色连续段。将这个结构翻译成生成函数：前者是 mx，后者是 $\frac{-mx^k}{1+\sum_{i=1}^{k-1}x^i}$（注意每个钦定带来 -1 的系数）。最后再套一层序列就得到

$$\left(1-mx-\frac{-mx^k}{1+\sum_{i=1}^{k-1}x^i}\right)^{-1}=\frac{1-x^k}{1-mx+(m-1)x^k}.$$

然后可以得到和做法二相同的递推。

做法四. 考虑先枚举所有极长连续段的长度 $l_1,\cdots,l_t$，其贡献为 $m(m-1)^{t-1}$。这样我们也可以直接写出生成函数

$$\frac{m}{m-1}\cdot\frac{1}{1-(m-1)\sum_{i=1}^{k-1}x^i}=\frac{m}{m-1}\cdot\frac{1-x}{1-mx+(m-1)x^k}$$

这个生成函数和做法三中的看起来有微妙的不同，这是 f_0 导致的。事实上，做法三中的生成函数才是完全正确的，这里这个会导致 $f_0=m/(m-1)$。不过，把上面那个分式化成真分式后就会发现其他项都是相同的。

做法五. 在得到生成函数后也可以直接展开：

$$\begin{aligned}
&[x^n]\frac{1-x^k}{1-mx+(m-1)x^k}=[x^n]\frac{1}{1-mx}\cdot\frac{1-x^k}{1+\frac{(m-1)x^k}{1-mx}}\\
&=[x^n]\frac{1-x^k}{1-mx}\sum_{i\ge0}(-1)^i\left(\frac{(m-1)x^k}{1-mx}\right)^i\\
&=\sum_{i\ge0}(-1)^i(m-1)^i[x^n]\frac{(1-x^k)x^{ik}}{(1-mx)^{i+1}}\\
&=\sum_{i\ge0}(-1)^i(m-1)^i\left([x^{n-ik}]\frac{1}{(1-mx)^{i+1}}-[x^{n-(i+1)k}]\frac{1}{(1-mx)^{i+1}}\right)\\
&=\sum_{i\ge0}(-1)^i(m-1)^i\left(m^{n-ik}\binom{n-ik+i}{n-ik}-m^{n-(i+1)k}\binom{n-(i+1)k+i}{n-(i+1)k}\right)
\end{aligned}$$

$i > n/k$ 后的项都是 0。每一个组合数都可以 $O(i) = O(n/k)$ 计算，总复杂度 $O((n/k)^2)$。

上面这种手法是常用的，即把分式写作 $\frac{1}{A(x)-B(x)} = \frac{1}{A(x)}\frac{1}{1-B(x)/A(x)}$ 然后展开。

例 5.9.（困难）禁止子串. P1393 Mivik 的标题：长为 n 的字符集大小为 m 的字符串，不能出现子串 S，求方案数

解答. 前面的容斥做法可以扩展到一般的禁止子串 S，只需要重新刻画由若干个间距小于 $k = |S|$ 的钦定位置而连成的一个大固定串。这样的东西形如 S 后面跟着若干个 S 的长度小于 k 的周期。所以总的生成函数就是

$$\left(1 - mx - \frac{-x^k}{1 + c(x)}\right)^{-1},$$

其中

$$c(x) = \sum_{i=1}^{k-1} [i \text{ 是 } S \text{ 的周期}] x^i.$$

例 5.10. P6076 [JSOI2015] 染色问题

例 5.11. P4491 [HAOI2018] 染色

5.3.5　格路径

例 5.12. 从 $(0, 0)$ 走到 (n, m)，只能向上向右走，求方案数

解答. 由于必须恰好有 n 个向右的步和 m 的向上的步，任意安排其顺序，方案数是 $\binom{n+m}{n}$。这也展示了组合数的对称性。另外，如果我们枚举首次走到 $x = n$ 时所在的 y 坐标 i，就可以得到上指标求和或者平行求和

$$\sum_{i=0}^{m} \binom{n-1+i}{i} = \binom{n+m}{m} = \binom{n+m}{n} = \sum_{i=0}^{m} \binom{n-1+i}{n-1}.$$

（当然也可以枚举首次走到 $y = m$ 时的 x 坐标）

例 5.13. 从 $(0, 0)$ 走到 (n, m)，可以向上向右向右上走，求方案数

解答. 枚举向右上走的步数 i，那么还需要 $n - i$ 个向右的步和 $m - i$ 个向上的步，任意安排其顺序，方案数是

$$\sum_i \binom{n+m-i}{i, n-i, m-i}$$

评注. 这两个问题的二元生成函数都非常好写，分别是 $\frac{1}{1-x-y}$ 和 $\frac{1}{1-x-y-xy}$。对其分析可以得到更多有用的结果。

评注. 把这两个问题顺时针旋转 45°，看看得到的问题是什么。

例 5.14. 从 $(0,0)$ 走到 (n,m)，只能向上向右走，且任一时刻不能跨过直线 $y=x$，求方案数

解答. 假设 $m \le n$，否则答案总是 0。我们用总的减去不合法的，而不合法的路径相当于是**碰到**直线 $y=x+1$ 的路径。对于这样一条路径，我们把路径中**首次**碰到直线之后的部分沿着 $y=x+1$ 对称，这样我们就得到了一条从 $(0,0)$ 到 $(m-1,n+1)$ 的路径。反过来，考虑任何一条从 $(0,0)$ 到 $(m-1,n+1)$ 的路径，这样的路径必然穿过直线 $y=x+1$。做同样的对称操作，我们就得到了一条从 $(0,0)$ 走到 (n,m) 且碰到 $y=x+1$ 的路径。也就是说这种变换是一个双射，从而不合法的路径数量就等于从 $(0,0)$ 走到 $(m-1,n+1)$ 的路径数量，即 $\binom{n+m}{m-1}$。

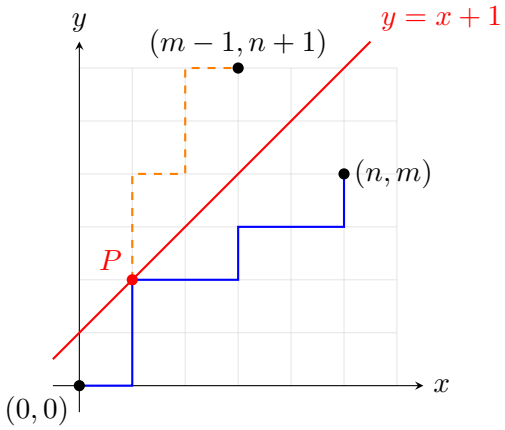


图 5.2: 关于 $y=x+1$ 对称

评注. 在顺时针旋转 45° 再关于 x 轴镜像后的图上想象这个过程或许会更容易。此时，不合法的路径是不碰到直线 $y=-1$ 的路径。

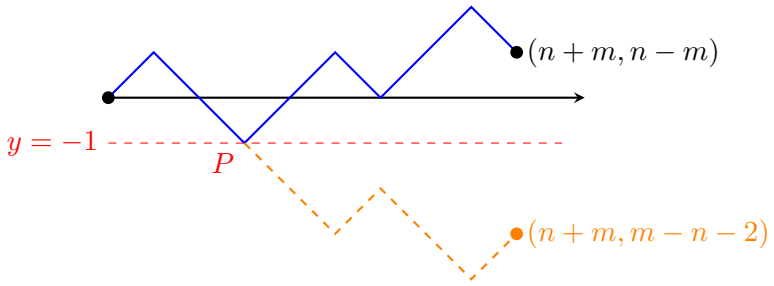


图 5.3: 关于

$y=-1$

对称

另外，这个容斥也可以推广到不碰到两条线的情况。

评注. 这个问题的一个（困难的）推广是 *Dyck* 路：从 $(0,0)$ 走到 (n,m)，不跨过直线 $y=kx+b$ 的方案数。

练习 5.15. 从 $(0,0)$ 走到 $(p+q-1, n+m-1)$，只能向上向右走，要求首次走到 $x=p$ 时所在的 y 坐标 $<m$。另一方面，这个要求等价于首次走到 $y=m$ 时所在的 x 坐标 $\geq p$。据此得到一个奇妙的恒等式。

5.3.6　卡特兰数

在例 5.14 中考虑 $n=m$ 的特殊情况，此时方案数是

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{2n+1}\binom{2n+1}{n}.$$

这个数列被称为**卡特兰数**（Catalan Number），前几项是 1, 1, 2, 5, 14, 42。它也在很多其他问题中自然地出现：

- 长为 $2n$ 的合法括号序列数量
- n 个 1 和 n 个 -1 组成的满足所有前缀和都非负的序列数量
- n 个点的儿子有序的二叉树数量
- $n+1$ 个点的儿子有序的多叉树数量
- n 个元素的出栈序数量
- $n+2$ 边形的三角剖分数量

括号序列

一个括号序列长什么样子？我们考虑所有最外层的括号，就得到 $\mathcal{A} = (\mathcal{A})(\mathcal{A})\cdots(\mathcal{A})$。或者，我们可以把除了第一个括号以外的看成一体，就得到 $\mathcal{A} = \{\epsilon\} + (\mathcal{A})\mathcal{A}$（$\epsilon$ 代表空序列）。把这两个构造翻译成递推式就得到

$$C_n = \sum_{k\geq 0} \sum_{(i_1+1)+\cdots+(i_k+1)=n} C_{i_1}\cdots C_{i_k}, \quad C_n = [n=0] + \sum_{i+1+j=n} C_i C_j.$$

更好的做法是翻译成生成函数：

$$C(x) = \frac{1}{1-xC(x)}, \quad C(x) = 1 + xC(x)^2.$$

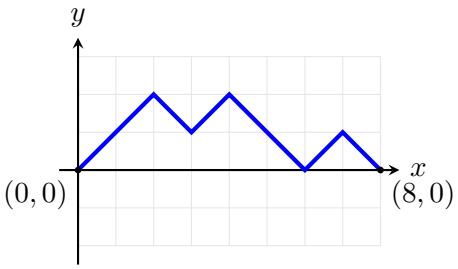
也可以翻译成区间 dp，比如 [CSP-S 2021] 括号序列（不过这个题稍微复杂一点点）。

± 1 序列

我们把 (看成 $+1$，) 看成 -1，那么括号序列就一一对应由 n 个 $+1$ 和 n 个 -1 组成的序列。这个转换是非常常用的。

定理 5.16. 括号序列合法当且仅当对应的 ± 1 序列的任何前缀和都非负。

这样我们就建立了这两个组合类之间的双射。另一方面，我们也经常把数列按照如下方式转化成格路径：从 $(0,0)$ 出发，逐个考虑 $i=1,\ldots,n$，遇到 a_i 就走一步向量 $(1,a_i)$。例如，$+1,+1,-1,+1,-1,-1,+1,-1$ 对应下图所示的路径：



记 s_i 为数列的前缀和，则容易发现 $x=i$ 处的高度为 s_i。这也说明了图上两个点 (i,s_i) 和 (j,s_j) 之间的高度差对应数列的区间和 $s_j-s_i=\sum_{j<k\le i}a_k$。而所有前缀和都非负对应着不触碰 $y=-1$。这样，我们就建立起了 ±1 序列与先前讨论的格路径之间的双射。

评注. 通过前面两个双射，我们证明了括号序列与 ±1 序列的数量都是卡特兰数，所以先前对括号序列建立的递推式也就对应着卡特兰数的递推式。

然而不同的组合结构带给人的直观感受是不同的！对于 ±1 序列，我们几乎没有任何直观；对于括号序列，我们有序列分解的直观；对于格路径，我们有图像的直观。另外，前一页中的格路径似乎比之前旋转 45° 之前的格路径更加直观。

事实上，我们也很容易对格路径建立先前对括号序列建立的递推：只需要考虑每一次触碰 $y=0$ 而分成的若干段即可。

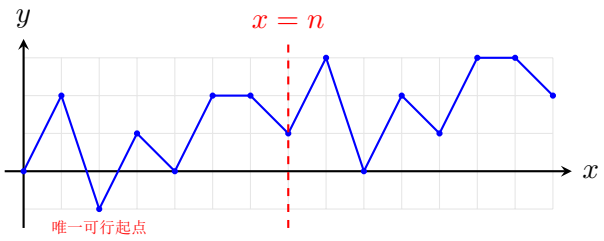
对于 ±1 数列，我们还有另一个方法计数，即 Raney 引理：

引理 5.17. Raney 引理. 对于一个和为 1 的整数序列，其所有循环移位中恰好有一个满足：所有前缀和都是正的

它有一个（我们暂时用不到的）推广如下：

引理 5.18. 推广的 Raney 引理. 对于一个和为 ℓ 且所有元素都 ≤ 1 的整数序列，其所有循环移位中恰好有 ℓ 个满足：所有前缀和都是正的

证明. 循环移位可以通过把序列复制一遍来处理。也就是说如果我们令 $a_{n+i}=a_i$，那么循环移位 t 后得到的序列就是 $a[t+1,n+t]$。我们现在可以把复制一遍的序列画成格路径：



现在，$[i+1,n+i]$ 这一段区间对应的格路径相当于以 (i,s_i) 为起点、以 $(n+i,s_{n+i})$ 为终点的格路径，而所有前缀和均为正相当于过程中高度始终高于起点。那么显然有且仅有以那个最小的 s_i（相等的话认为 i 大的小）为起点才能做到这一点。推广的结果也可以类似证明。 □

我们需求的是所有前缀和非负，而 Raney 引理中是所有前缀和 >0。我们可以通过在序列开头加一个 1 来处理这个问题。也就是说我们考虑双射：n 个 $+1$ 和 n 个 -1 组成的所有前缀和非负的序列 $\leftrightarrow$ 由 $n+1$ 个 $+1$ 和 n 个 -1 组成的所有前缀和 >0 的序列。

对于新的问题，Raney 引理告诉我们可以将所有可能的 $\binom{2n+1}{n}$ 个序列按照循环移位划分等价类，而每个等价类恰好贡献一种合法方案。由于 $\gcd(n+1,n)=1$，每个等价类的大小都是 $2n+1$（也就是说一个序列的所有循环移位都是不同的），除法原理就告诉我们合法方案数为

$$\frac{1}{2n+1}\binom{2n+1}{n}.$$

二叉树

二叉树具有自然的左子树-根-右子树结构，也就是 $\mathcal{A}=\{\epsilon\}+\begin{array}{ccc}&\bullet&\\ /&&\backslash\\ \mathcal{A}&&\mathcal{A}\end{array}$，所以二叉树和合法括号序列有相同的递推式。又因为初值也相同，所以二叉树的数量也是卡特兰数。

我们也可以把这个结构与括号序列的结构进行对应，据此得到一个双射：记 $S[T]$ 为树 T 映射到的括号序列，则

$$S\left[\begin{array}{ccc}&\bullet&\\ /&&\backslash\\ L&&R\end{array}\right]=(S[L])S[R],\quad S[\epsilon]=\epsilon.$$

另外，我们还需要注意 n 个点的二叉树与有 n 个**内点**的满二叉树[1]的一一对应（内点即左右子树都非空的节点）：想象在每个非内点下面长叶子来变成内点

多叉树

通过枚举儿子数量，我们得到

$$\mathcal{A}=\bullet+\begin{array}{c}\bullet\\ |\\ \mathcal{A}\end{array}+\begin{array}{ccc}&\bullet&\\ /&&\backslash\\ \mathcal{A}&&\mathcal{A}\end{array}+\begin{array}{ccccc}&&\bullet&&\\ &/&&\backslash&\\ \mathcal{A}&&\mathcal{A}&&\mathcal{A}\end{array}+\cdots$$

那么具有 $n+1$ 个点的多叉树和长为 $2n$ 的括号序列有相同的递推式。我们同样可以据此得到一个双射：

$$S\left[\begin{array}{ccccc}&&\bullet&&\\ &/&\cdots&\backslash&\\ T_1&&\cdots&&T_k\end{array}\right]=(S[T_1])\cdots(S[T_k])$$

出栈序

维护一个栈 S 和一个整数 $i=1$，在任意时刻可以 (i) 如果 $i\le n$，则将 i 入栈，并令 $i\leftarrow i+1$ 或者 (ii) 如果栈非空，则弹出当前栈顶。等到无法继续操作时，按照出栈的顺序可以得到一个排列。这就是 $1,\ldots,n$ 的出栈序。这是一个不太常用的模型，因为它不够直观而且分析比较复杂。

[1]这里满二叉树指所有非叶结点都有两个儿子的二叉树

考虑最后一次操作，这一定是一个出栈。假设这次出栈的元素是 x，那么考虑 x 的入栈时刻 t，则在 t 时刻结束后栈中一定只有 x 自己（否则 x 无法最后出栈）。因此，出栈序可以按顺序划分成三部分：$[1,x)$ 的出栈序、$(x,n]$ 的出栈序、x。这又是一个卡特兰数类型的递归。

另一方面，我们可以注意到操作序列与出栈顺序的联系。由于每个元素出入栈各一次，序列的长度必然是 $2n$。因此，上面的分析也给出了操作序列的划分，于是出栈序与操作序列一一对应。而操作序列天然对应一个括号序列（入栈/出栈分别对应左/右括号）。

按照前面的分析，我们还可以给出和二叉树的一个双射：我们给二叉树按照中序遍历编号，则出栈序对应一个后序遍历。我们知道中序遍历 + 先/后序遍历可以唯一确定一棵二叉树。

最后，出栈序等价于 312 禁子排列，即不存在 $i<j<k$ 但是 $\pi_j<\pi_k<\pi_i$ 这种模式的排列。因为如果我们考虑 π_n，则 $k=n$ 的限制的影响是 $(\pi_n,n]$ 必须出现在 $[1,\pi_n)$ 右边，然后剩下的限制只有这两块内部的限制。

多边形三角剖分

顺时针给所有顶点编号。考虑最终剖分中 $1-n$ 这条边在其所在的三角形中的对点 i，则此三角形划分出了两个子结构，规模分别为 i 和 $n-i+1$。这是 C_{n-2} 满足的递归。

可以据此设计到二叉树的双射：每个三角形视为一个点，与相邻的三角形连边。事实上，这就是平面图转对偶图。

5.4　根据组合结构 dp

组合结构往往具有自然的子结构。最常见的包括：

- 序列：前缀/区间
- 树：子树/链
- 集合：子集
- 网格：1/2-side 子矩形；轮廓线
- DAG：子 DAG（拓扑序）

例 5.19. CSP-S 2021. 括号序列

5.5　利用乘法原理 dp

合适地刻画子问题，使得每种子问题对后续状态的转移系数都相同。

例 5.20. CSP-S 2025. 员工招聘

解答. 做法零：将 (前 i 个人，要求形成的部分排列为 P，的部分排列数量) 作为子问题。$\tilde{O}(n!)$

做法一：将 (前 i 个人，要求构成的集合为 S、目前未录用的人数为 j，的部分排列数) 作为子问题。$\tilde{O}(2^n)$。这个刻画对大部分排列类计数都适用。

做法二（伪）：将 (前 i 个人，要求目前未录用的人数为 j、$c>j$ 的人中还没放的人数为 k，的部分排列数) 作为子问题。然而，这样不能转移，因为无法知道下一个位置不录用时再下一个位置的候选人数会如何变化。

做法二（真）：上一个做法中，“无法知道下一个位置不录用时再下一个位置的候选人数会如何变化”的原因如下：假设目前不录用人数是 j，而之前可能已经录用了一些 $c=j+1$ 的人，但这个人数没有在状态中记录。所以我们可以考虑把 $c=j+1$ 的人等到最后再一起放了。也就是将 (前 i 个人，要求目前未录用的人数为 j、$c>j$ 的人中还没放的人数为 k、只确定了 $c\le j$ 的人的位置，形成的部分排列数) 作为子问题。这个技巧也称为“贡献延后计算”。

另一个视角. 假设我们先钦定了所有位置的录取情况，那么如何计算方案数？对于一个录取的位置，必须有 $s_i=1$ 且 $c_{\pi_i}>f_i$，其中 f_i 为 i 前面的未录取人数；对于一个未录取的位置，需要 $s_i=0$ 或者 $c_{\pi_i}\le f_i$。那么相当于每个位置有限制 $c_{\pi_i}>f_i$ 或者 $c_{\pi_i}\le f_i$ 或者无限制。

现在来考虑子问题：(i) 如果只有某一种限制怎么做？(ii) 如果只有某两种限制怎么做？(iii) 怎么把某一种限制用另外两种限制表示？

如果把无限制表示成 $[c_{\pi_i}>f_i]+[c_{\pi_i}\le f_i]$，那就能得到上一页中的做法。如果把 $[c_{\pi_i}>f_i]$ 表示成 $1-[c_{\pi_i}\le f_i]$，那么就得到另一种做法，也就是容斥。

最后，可以在 dp 中枚举所有位置的录取情况。

5.6　生成函数初步

这里主要介绍怎么用生成函数（Generating Function, GF）处理序列（假设已经得到序列的某些性质，如递推关系等）。事实上更多情况下可以直接对着问题的结构写出生成函数，而无需额外经过一步 dp 转化。绝大多数的推式子问题都有办法用生成函数处理。但需要强调的是生成函数**并非万能**，不应忽视其他的技巧和组合意义。在使用生成函数之前先做一些基本的处理往往能简化推导。

最基本和主要的手段就是把需要处理的序列 (f_i) 挂到幂级数 $\sum_{i\ge0}f_ia_ix^i$ 上去，其中 a_i 是自己选定的另一组系数，通常是 1（普通生成函数，**O**rdinary GF）或 $1/i!$（指数生成函数，**E**xponential GF）。这样导出的幂级数几乎总是比序列容易处理的。注意在实践中一般需要假设 $x\approx0$ 是一个小量。我们后面说的“幂级数”和“生成函数”都是指的同一个东西。

记 $[x^n]F(x)$ 为幂级数 $F(x)$ 的第 n 次项系数。在上下文清楚时，有时我们也用 f_n 代表这个事情。（注意：如果上面挂到幂级数上时使用了非 1 的 a_i，那么这里的 f_n 代表上面 $\sum_{n\ge0}f_na_nx^n$ 中的 f_na_n 而不是 f_n 本身，后同）

5.6.1　运算法则 I

可以对幂级数与它的封闭形式做同样的运算来得到以下的对应关系：

线性运算 $H(x)=aF(x)+bG(x)$ 对应 $h_n=af_n+bg_n$

卷积 $H(x)=F(x)G(x)$ 对应 $h_n=\sum_{i=0}^n f_ig_{n-i}$。更一般地，

$$[x^n]F_1(x)F_2(x)\cdots F_m(x)=\sum_{i_1+i_2+\cdots+i_m=n}([x^{i_1}]F_1(x))([x^{i_2}]F_2(x))\cdots([x^{i_m}]F_m(x)).$$

求导 $H(x)=F'(x)$ 对应 $h_n=(n+1)f_{n+1}$。

5.6.2　导数

本节给不熟悉导数的同学提供

导数有几个不同的记号，$f'(x)$、$\frac{\mathrm{d}}{\mathrm{d}x}f(x)$、$\mathbf{D}f(x)$ 都可以用来代表求导。如果需要反复求导（高阶导数），那么可以用 $f'''(x)$、$f^{(k)}(x)$、$\frac{\mathrm{d}^k}{\mathrm{d}x^k}f(x)$、$\mathbf{D}^k f(x)$ 这些记号。

我们做的所有操作都是**形式上的**。你只需要（熟练）记住以下这些事实：

- 运算性质 $(af(x)+bg(x))' = af'(x)+bg'(x)$，$(f(x)g(x))' = f'(x)g(x)+f(x)g'(x)$，$(f(g(x)))' = f'(g(x))g'(x)$
- 初等函数的导数 $(x^\alpha)' = \alpha x^{\alpha-1}$，$(\ln x)' = x^{-1}$，$(e^x)' = e^x$

下面是几个练习。这些结果也是有价值且值得记住的：

- 常数的导数是 0
- $(f(x)/g(x))' = \frac{f'(x)g(x)-f(x)g'(x)}{g(x)^2}$
- $(\log f(x))' = \frac{f'(x)}{f(x)}$
- $(e^{f(x)})' = f'(x)e^{f(x)}$
- $(fg)^{(k)} = \sum_i \binom{k}{i} f^{(i)} g^{(k-i)}$
- $(f_1 f_2 \cdots f_k)' = (f_1 f_2 \cdots f_k)\sum_i \frac{f_i'}{f_i}$

评注. 求导 $\mathbf{D}$ 是一个**算子**，也就是一个函数到函数的映射。我们可以做一些简单的算子之间的运算，如 $\mathbf{D}^2$、$\mathbf{D}+1$、$x\mathbf{D}$ 等。这里，1 和 x 分别代表 $f \mapsto f$ 和 $f \mapsto xf$ 这两个算子，乘积代表复合。也就是说，$\mathbf{D}^2 f = f''$，$(\mathbf{D}+1)f = f'+f$，$(x\mathbf{D})f = xf'$。

由于映射满足结合律，算子的乘积自然也满足结合律。但是，交换律则一般不满足。例如，$(\mathbf{D}x)f = (xf)' = f + xf' = 1 + x\mathbf{D}$。

5.6.3　运算法则 II

$H(x) = x^m F(x)$ 对应移位 $h_n = f_{n-m}$。注意：尽量不要让你的式子中出现**负下标项**，所以在 m 为负时要小心。。

$H(x) = \frac{1}{1-x}F(x)$ 对应前缀和 $h_n = \sum_{i\le n} f_i$

$H(x) = (1-x)F(x)$ 对应差分 $h_n = f_n - f_{n-1}$

$H(x) = \vartheta F(x) \stackrel{\text{def}}{=} xF'(x)$ 对应原地的求导 $h_n = nf_n$

$H(x) = F(x) \bmod (x^k - 1)$ 对应 $h_r = \sum_n f_{kn+r}$。（事实上模一般的 $Q(x)$ 相当于按一个线性递推累加系数）

$H(x) = x^k F^{(k)}(x)$ 对应 $h_n = n^{\underline{k}} f_n$

$H(x) = (x\mathbf{D})^k F(x)$ 对应 $h_n = n^k f_n$。然而，$(x\mathbf{D})^k$ 一般没有好的封闭形式（虽然 $(x\mathbf{D})^k = \sum_i \left\{ {k \atop i} \right\} x^i \mathbf{D}^i$）

评注. 这里是一些记忆的提示。首先，从生成函数操作到数列的操作不需要单独记忆，因为我们可以做完所有的生成函数操作后用 $[x^n]F(x)$ 这个运算来提取系数，提取系数时一般只需要将生成函数的封闭形式重新展开成幂级数。

对于数列操作到生成函数操作，可以把操作分成三类：线性与卷积运算、让 f_n 乘一个 n 的多项式、同余求和。对于第一类运算，可以通过“乘上 x^n 然后对 n 求和”的方式从数列恒等式得到生成函数；对于第二类运算，需要记住高阶导数会导出 k 次下降幂的形式，然后配合线性运算和移位凑出想要的多项式；对于最后一类运算，熟悉这个形式即可。

5.6.4　最重要的一些生成函数

下面几个恒等式是应该（双向）记住的，因为它们实在是太常见了。

$$(1+x)^\alpha=\sum_{n\geq 0}\binom{\alpha}{n}x^n,\quad \ln\left(\frac{1}{1-x}\right)=\sum_{n\geq 1}\frac{x^n}{n}\quad e^x=\sum_{n\geq 0}\frac{x^n}{n!}$$

第一个恒等式——也就是二项式定理——的一些特殊情况也是值得记忆的（证明它们！）：

$$\frac{1}{(1-x)^k}=\sum_{n\geq 0}\binom{n+k-1}{n}x^n,\quad \frac{x^k}{(1-x)^{k+1}}=\sum_{n\geq 0}\binom{n}{k}x^n,\quad \frac{1}{\sqrt{1-4x}}=\sum_{n\geq 0}\binom{2n}{n}x^n$$

代入

x 可以代入为任何 ≈ 0 的东西（有些时候还可以代入别的东西[2]，取决于**收敛半径**）。在生成函数的语境下，≈ 0 的东西指的就是任何**常数项为 0** 的幂级数。

直接的例子：

$$F(x^k)=\sum_{n\geq 0}f_nx^{nk},\quad F(rx)=\sum_{n\geq 0}f_nr^nx^n.$$

复杂一点的例子：

$$\sum_{n\geq 0}\left(\frac{x}{1-x}\right)^n=\frac{1}{1-\frac{x}{1-x}}=\frac{1-2x}{1-x}$$

代值的例子：把 $x=1$ 和 $x=-1$ 分别代入 $(1+x)^n$。把 $|r|<1$ 的 r 代入 $1/(1-x)^k$。

5.6.5　用生成函数处理序列

化简范德蒙德卷积 $\sum_i\binom{r}{i}\binom{s}{n-i}$。我们只需要乘上 x^n 然后对 n 求和，化简得到的生成函数，最后再提取 $[x^n]$。也就是

$$[x^n]\sum_n x^n\binom{r}{i}\binom{s}{n-i}.$$

用这个方法化简（注意：你需要决定对哪个变量做我们前面说的事情）

$$\sum_k\binom{l}{m+k}\binom{s+k}{n}(-1)^k,\quad \sum_{-q\leq k\leq l}\binom{l-k}{m}\binom{q+k}{n},\quad \sum_{k\geq 0}\binom{n+k}{m+2k}\binom{2k}{k}\frac{(-1)^k}{k+1}.$$

（然而问题 8 没有办法直接这样做... 但是有另外的技巧可以做...）

试着对下面的式子做同样的事情（我们一会再来教大家怎么提取这种系数）

$$\sum_{k\leq n}\binom{n-k}{k}(-1)^k$$

也可以用生成函数处理递推式来得到方程。

例 5.21. 对斐波那契数列的递推公式 $f_n=f_{n-1}+f_{n-2}$ 使用前面的方法，根据得到的方程解出生成函数。然后，通过对生成函数求导，导出一个 $\sum_{i=0}^n f_if_{n-i}$ 的简化形式。

[2]在 OI 里，你基本可以认为是任何东西

练习 5.22. 对卡特兰数的递推公式 $C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}$ 使用前面的方法。据此导出卡特兰数的封闭形式。

练习 5.23. 对错排的递推公式 $d_n = (n-1)(d_{n-1} + d_{n-2})$、$d_n = nd_{n-1} + (-1)^n$ 使用前面的方法（试试 OGF 和 EGF，也就是分别乘 x^n 和 $x^n/n!$）。

现在，我们看看如何用生成函数处理以前的一些问题。这里，我们可以先只关心生成函数的封闭形式，等会再考虑如何计算。

$$\sum_{0\le i\le n} \mathrm{Fib}_i, \qquad \sum_{1\le i_1\le i_2\le\cdots\le i_m\le n} 1, \qquad \sum_{i=0}^{n} r^i i^k, \qquad \sum_i \binom{n}{i} r^i a_{i \bmod k}.$$

练习 5.24. 使用生成函数表示

$$\sum_i \binom{n}{i} r^{\lfloor i/k \rfloor},$$

从而得到 $r^{1/k}$ 和 ω_k 都不存在时的一个做法。

感觉是不太广为人知的东西。

评注. 可以使用一些深刻的背景知识将其扩展到 $\langle H(x), F(x)\rangle$，其中 H 是某个稀疏多项式，$F(x)$ 是任意线性递推，$\langle \cdot, \cdot \rangle$ 表示系数点积。

5.6.6　计算生成函数

得到一个生成函数的封闭形式后，如何用于计算第 n 项或者前 n 项？下面是几种常见的方法：

- 对于比较简单的情况，可以直接使用前面的常见级数展开来得到系数。
  - 这里可能能直接得到封闭形式，也可能得到一个卷积求和
- 对于有理分式，可以分式分解，或者使用线性递推
- 使用 FFT 来进行各种多项式运算
- 使用暴力来进行各种多项式运算
- 通过求导等手段得到递推式
- 拉格朗日反演

分式分解

对于 $P(x)/Q(x)$（P, Q 都是多项式）的形式，可以先将其变成 $P(x) \operatorname{div} Q(x) + \frac{P(x) \bmod Q(x)}{Q(x)}$。然后可以假设 $\deg P < \deg Q$。（有人不会多项式除法吗？）

真分式总是有如下的分解：

$$\frac{P(x)}{\prod (a_i x + b_i)^{k_i}} = \sum_i \sum_{j=1}^{k_i} \frac{c_{ij}}{(a_i x + b_i)^j}.$$

可以待定系数计算 c_{ij}。当 $k_i = 1$ 时，有更方便的算法。两侧同乘 $a_i x + b_i$，然后代入 $x_i = -b_i/a_i$ 就得到

$$\frac{P(x_i)}{\prod_{j\ne i}(a_j x_i + b_j)^{k_j}} = c_{i1}$$

例 5.25. 线性递推. 对于有理真分式 $F(x)=P(x)/Q(x)$，两侧乘 $Q(x)$ 得到 $F(x)Q(x)=P(x)$。展开卷积就得到线性递推。

使用分式分解即可得到所有线性递推的相关结论。例如，你现在应该能够自行推出

$$f_n=\sum_{i=1}^{k}a_if_{n-i}+r^n(\sum_{i=0}^{m}b_in^i)$$

的生成函数的封闭形式。

评注. 我们现在有非常简单的 $O(\mathbf{M}(m)\log n)$ 算法来得到一个有理分式的第 n 次项系数：注意到 $Q(x)Q(-x)$ 只有偶数项有值，考虑 $[x^n]\frac{P(x)}{Q(x)}=[x^n]\frac{P(x)Q(-x)}{Q(x)Q(-x)}$，后者只与分子的奇数/偶数次项有关（取决于 $n \bmod 2$），据此我们可以将 n 减半，而分式的规模保持不变。

$O(n^2)$ 多项式运算

下面的 n,m 都指多项式的**项数**而不是次数。对 $F(x)$ 用 n，对 $G(x)$ 用 m。

- 可以在线性时间内计算 $aF(x)+bG(x)$、$x^mF(x)$、$F'(x)$、$\int_0^x F(t)\mathrm{d}t$。
- 可以在 $O(nm)$ 时间内计算 $F(x)G(x)$。所以当 $\min(n,m)$ 非常小时复杂度优秀
- 可以在 $O(m(\deg F-\deg G))$ 时间内计算 $F(x)$ div $G(x)$ 和 $F(x) \bmod G(x)$
- 后两者可以用 FFT 做到 $O(N\log N)$，其中 $N=\max(\deg F,\deg G)$

假设 F 的**项数** $<m$，那么可以在 $O(nm)$ 时间内计算下列级数的前 n 项（可以假设 $m\le n$，为什么？）。实践中 m 经常很小，此时这种暴力的复杂度非常优秀。

- $G=F^{-1}H$（需求 $f_0\neq 0$）：也就是 $G\cdot F=H$，直接展开卷积就得到一个递推式
- $G=\ln F$（需要 $f_0=1$）：两侧求导得到 $G'=F'/F$，也就是 $FG'=F'$，展开卷积就得到一个递推式。初值 $g_0=0$。
- $G=e^F$（需要 $f_0=0$）：两侧求导得到 $G'=F'G$，展开卷积得到递推式。初值 $g_0=1$。
- $G=F^k$：两侧求导得到 $G'=kF'G/F$，也就是 $FG'=kF'G$，展开卷积得到递推式。初值 $g_0=f_0^k$。
- 练习：假设 F 是分子分母项数都不超过 m 的有理分式，在 $O(nm)$ 时间内求出 F 的前 n 项。

以上都可以用 FFT 做到 $O(n\log n)$。注意到在 m 为常数时，这还不如上面的做法。

5.6.7　更多技巧

还有值得再讲一个专题的例子和技巧，然而这里时间不够。在生成函数专题（如果有）会详细展示。

本章总结

- 二项式系数的定义和基本恒等式：《具体数学》表 5-4
- 常用的组合计数模型：球盒问题、变量计数、排列计数、数列计数、格路径计数
- 卡特兰数相关模型：括号序列、+1 序列、二叉树、多叉树、出栈序、多边形三角剖分
- 生成函数初步：生成函数的概念、生成函数运算与数列运算的对应、求导的方法和技巧、最基本的几个幂级数和封闭形式、通过乘 x^n 然后对 n 求和来得到生成函数、通

过幂级数展开将封闭形式的生成函数还原回数列、通过生成函数方程得到数列递推式、分式分解与线性递推

参考文献

本讲义主要由我个人写作，整理了许多我在 OI 中学到的技巧。以下是一些过去，现在和未来参考的资料。

Gemini 3 (Thinking) 帮助我寻找了一些例题、补充了一些内容，并教会了我一些我不甚熟悉的内容。事实上，我认为 AI 在本讲义中的任何一个具体部分上的能力都远远强于我，但在对知识的宏观统筹能力上还有所欠缺。

EI 老师的博客（CSDN，洛谷，博客园）。EI 老师极大地推动了 OI 数数技术的发展。

command_block 的洛谷博客和知乎文章。